FIAP ON

DIGITAL BUSINESS ENABLEMENT

WEB SERVICES RESTFUL (JAX-RS, JSON)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 3.1 – Retornando usuários – API fiap-on SandBox. 10
Figura 3.2 – Retornando usuário de id 1 – API fiap-on SandBox. 11
Figura 3.3 – JSON trafegando via HTTP/HTTPs. 13
Figura 3.4 – Retornando usuário de id 1 – API fiap-on SandBox. 19
Figura 3.5 – Visualizando HTTP Headers no Chrome - API fiap-on SandBox. 20
Figura 3.6– Criando uma Collection no Postman. 21
Figura 3.7 – Nomeando uma Collection no Postman. 22
Figura 3.8 – Criando uma requisição GET no Postman. 22
Figura 3.9 – Salvando a requisição GET na Collection Fiap Usuários. 23
Figura 3.10 – Criando uma requisição PUT no Postman. 24
Figura 3.11 – Enviando a requisição PUT no Postman. 24
Figura 3.12 – Criando e enviando uma requisição POST no Postman. 25
Figura 3.13 – Headers HTTP e Status Code da resposta ao POST no Postman. 26
Figura 3.14 – Criando e enviando uma requisição DELETE no Postman. 26
Figura 3.15 – Criando e enviando uma requisição de teste do DELETE no Postman. 27
Figura 3.16 – Enviando uma requisição GET de teste no Postman. 27
Figura 3.17 – Criando um Dynamic Web Project no eclipse. 40
Figura 3.18 – Configurando o Source folder no build path do projeto. 41
Figura 3.19 – Configurando o web module do projeto. 41
Figura 3.20 – Projeto UsuarioCrudRest no Projeto Explorer Eclipse. 41
Figura 3.21 – Adicionando o jar Oracle no projeto. 42
Figura 3.22 – Convertendo o projeto ClienteCrudSoap para Maven Project. 42
Figura 3.23 – Criando o arquivo pom do projeto. 42
Figura 3.24 – O arquivo pom.xml do projeto no Project Explorer do Eclipse. 43
Figura 3.25 – O arquivo pom.xml do projeto no Project Explorer do Eclipse. 45
Figura 3.26 – Criando a classe EntityManagerFactorySingleton. 47
Figura 3.27 – Criando a sequence SEQ_USUARIO no Oracle FIAP. 48
Figura 3.28 – O pacote br.com.fiap.controle.entity no Project Explorer do Eclipse. ..48
Figura 3.29 – Log de execução de teste do Hibernate no Console Eclipse. 51
Figura 3.30 – Confirmação de gravação no Oracle, via acesso pelo Oracle Developer. 51
Figura 3.31 – Criação de pacotes de persistência no Project Explorer do Eclipse....52
Figura 3.32 – Teste da API - /controle/ola. 55
Figura 3.33 – Teste da API - /controle/ola após ajuste em @Produces. 55
Figura 3.34 – Teste da API - /controle/usuarios – Accept application/json. 58
Figura 3.35 – Teste da API - /controle/usuarios – Accept application/xml. 59
Figura 3.36 – Teste da API - /controle/usuarios no Chrome. 59
Figura 3.37 – HTTP Request Headers no Postman. 62
Figura 3.38 – POST Request Body no Postman. 62
Figura 3.39 – Response Body e Status Code no Postman. 62
Figura 3.40 – Response Headers - Location no Postman. 63
Figura 3.41 – Status Code 409 Conflict no Postman. 63
Figura 3.42 – Testando uma solicitação PUT equivocada no Postman. 65
Figura 3.43 – Usuário criado via POST no Postman. 65
Figura 3.44 – Configurando Headers da requisição PUT no Postman. 66
Figura 3.45 – Configurando o Body e enviando uma requisição PUT no Postman. ..66

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Figura 3.46 – Diagrama de sequência – HTTP Basic Authentication. 67
Figura 3.47 – tomcat-users.xml em Servers no Project Explorer do Eclipse. 68
Figura 3.48 – web.xml no Project Explorer do Eclipse. 70
Figura 3.49 – Requisição GET – Basic Auth no Postman. 76
Figura 3.50 – Requisição GET – HTTP Headers com Basic Auth no Postman 77
Figura 3.51 – Teste de requisição GET – HTTP Headers no Postman. 77
Figura 3.52 – Teste de requisição GET com MemoryRealm no Mozilla Firefox. 77
Figura 3.53 – Teste de requisição GET sem Headers com MemoryRealm no Postman. 78
Figura 3.54 – Teste de requisição com Basic Auth no Postman. 78

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE QUADROS

Quadro 3.1 – Categorias de códigos de status para o protocolo HTTP 28

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 3.1 – Lista de usuários representado em JSON. 14
Código-fonte 3.2 – Lista de usuários representado em XML. 14
Código-fonte 3.3 – Comparações entre Arrays declarados em JSON e XML. 15
Código-fonte 3.4 – Lista de usuários em JSON sem o array usuários. 15
Código-fonte 3.5 – Mensagem de requisição HTTP.....17
Código-fonte 3.6 – Mensagem de resposta HTTP. 18
Código-fonte 3.7 – Exemplo de resposta JSON com HATEOAS. 32
Código-fonte 3.8 – Mock fiap-on SandBox. 39
Código-fonte 3.9 – pom.xml. 45
Código-fonte 3.10 – persistence.xml. 46
Código-fonte 3.11 – EntityManagerFactorySingleton.java. 47
Código-fonte 3.12 – Usuario.java. 50
Código-fonte 3.13 – Teste.java. 50
Código-fonte 3.14 – UsuarioDAO.java. 52
Código-fonte 3.15 – UsuarioDaoImpl.java. 52
Código-fonte 3.16 – web.xml. 53
Código-fonte 3.17 – ServicesConfig.java. 53
Código-fonte 3.18 – UsuarioService.java. 54
Código-fonte 3.19 – Usuarios.java. 56
Código-fonte 3.20 – UsuarioService.java. 57
Código-fonte 3.21 – @GET e @DELETE por id em UsuarioService.java. 60
Código-fonte 3.22 – @Post em UsuarioService.java. 61
Código-fonte 3.23 – @PUT em UsuarioService.java. 64
Código-fonte 3.24 – catch PersistenceException em GenericDAOImpl.java. 65
Código-fonte 3.25 – tomcat-users.xml. 69
Código-fonte 3.26 – server.xml. 69
Código-fonte 3.27 – web.xml. 71
Código-fonte 3.28 – security-constraint em web.xml. 71
Código-fonte 3.29 – security-role em web.xml. 72
Código-fonte 3.30 – login-config em web.xml. 72
Código-fonte 3.31 – ServicesConfig.java. 73
Código-fonte 3.32 – UsuarioService.java. 76

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

3 WEB SERVICES RESTFUL (JAX-RS, JSON) 8
3.1 O que é REST? 8
3.2 Restrições REST 9
3.2.1 Interface Uniforme 9
3.2.1.1 Crie soluções baseadas em recursos.....9
3.2.1.2 Manipule os recursos utilizando representações.....11
3.2.1.3 JSON.....12
3.2.1.4 Por que JSON? 14
3.2.1.5 JSON.....14
3.2.1.6 XML 14
3.2.1.7 Manipule os recursos por meio de operações padronizadas 16
3.2.1.8 HTTP Headers 16
3.2.1.9 Media Types.....19
3.2.1.10 HTTP Methods 20
3.2.1.10.1 GET 20
3.2.1.11 PUT 23
3.2.1.12 POST.....25
3.2.1.13 Delete 26
3.2.1.14 Respostas padronizadas 28
3.2.1.15 HTTP Status Code 28
3.2.1.16 Uso Geral 29
3.2.1.17 GET 30
3.2.1.18 POST ou PUT 31
3.2.1.19 PUT 31
3.2.1.20 DELETE 31
3.2.1.21 POST, PUT e DELETE.....31
3.2.1.22 Use mensagens autodescritivas.....31
3.2.1.23 Utilize HATEOAS.....32
3.2.2 Não mantenha o estado - Stateless 33
3.2.3 Utilize Cache 33
3.2.4 Utilize arquitetura Client-Server.....34
3.2.5 Arquitete o sistema em camadas 35
3.2.6 Forneça código sobre demanda.....35
3.2.7 Aplicações RESTful.....35
3.2.8 O Mock Fiap-on.....35
3.3 REST em java 39
3.3.1 JAX-RS.....39
3.3.2 Serviço Fiap-on em JAX-RS, Jersey e JPA.....40
3.3.2.1 Criando o Projeto 40
3.3.2.2 Convertendo o Projeto para Maven Project.....42
3.3.3 Adicionando as dependências Maven 43
3.3.4 Configurando o JPA 45
3.3.5 Criando o back-end do Serviço 47
3.3.6 Configurando o JAX-RS 53
3.3.7 Codificando e testando o Serviço de forma básica 54
3.3.8 Consultando todos os usuários – HTTP GET.....55
3.3.9 Consultando e excluindo usuários pelo id – HTTP GET e DELETE.....59

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3.3.10 Criando um usuário – HTTP POST 60
3.3.11 Alterando um usuário – HTTP PUT 63
3.3.12 Controlando o acesso à API 67
3.3.12.1 Tomcat Realm 68
3.3.12.2 Autenticação MemoryRealm 68
3.3.12.3 Autorização MemoryRealm 73
3.3.12.4 Testando Autenticação e Autorização 76
3.4 Considerações finais 78
REFERÊNCIAS 80
GLOSSÁRIO 82

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3 WEB SERVICES RESTFUL (JAX-RS, JSON)

3.1 O que é REST?

Representational State Transfer (REST) é um estilo arquitetônico que especifica restrições de design, para que aplicações que implementem esse estilo aproveitem a arquitetura Web em seu benefício. Ao se enquadrar em tais restrições, uma aplicação RESTful obterá propriedades desejáveis no ambiente Web, tais como, comunicação heterogênea entre soluções (com baixa complexidade semântica), maior performance, escalabilidade e facilidade de desenvolvimento e manutenção.

Traduzindo livremente “Representational State Transfer” ou REST, temos Transferência de Estado Representativo. O termo REST foi introduzido e definido em 2000, na tese de doutorado de Roy Fielding para a University of California, IRVINE.

Nessa tese, resumidamente, Roy primeiro determina uma metodologia de classificação para estilos arquiteturais, depois apresenta diversos estilos, cada um com o conjunto de regras que o define. Na sequência, demonstra os requisitos da arquitetura WWW e os problemas enfrentados na concepção e avaliação das melhorias propostas aos principais protocolos de comunicação web para, por fim, definir o estilo arquitetônico REST, como um modelo de como a Web moderna deveria funcionar para sistemas hipermídia distribuídos.

Fielding participou efetivamente no desenvolvimento de muitos dos protocolos Web, incluindo HTTP (HyperText Transfer Protocol) e URI (Uniform Resource Identifier), sendo assim, não é muito difícil compreendermos porque o estilo REST está tão alinhado à arquitetura Web.

Segundo Fielding, o REST fornece um conjunto de restrições arquitetônicas que, quando aplicadas como um todo, enfatizam a escalabilidade das interações dos componentes, a generalidade das interfaces, a implantação independentemente de componentes e componentes intermediários para reduzir a latência de interação, reforçar a segurança e encapsular sistemas legados.

Então, antes de desenvolvermos serviços RESTful em Java, vamos conhecer os itens que compõem esse conjunto de restrições e descobrir como cada um ajuda as aplicações RESTful a serem simples, leves e rápidas.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3.2 Restrições REST

3.2.1 Interface Uniforme

A restrição de interface uniforme define a interface entre clientes e servidores. Essa restrição é muito importante e muitas vezes a mais citada nos materiais e documentos sobre REST. Os princípios contidos nessa restrição simplificam e desacoplam a arquitetura, permitindo que cada parte evolua de forma independente.

Os princípios orientadores da interface uniforme estão elencados a seguir.

3.2.1.1 Crie soluções baseadas em recursos

Um recurso é um elemento abstrato que nos permite mapear qualquer informação do mundo real, como um elemento para acesso via Web. Uma solução RESTful, geralmente, expõe um conjunto de recursos que identificam os objetivos da interação entre o serviço e os seus consumidores.

Os recursos são identificados e acessados por URIs que compõem um namespace global e único. Nesse contexto, os próprios recursos são separados das representações que serão devolvidas ao cliente consumidor. Por exemplo, imagine que queremos acessar uma lista de usuários (informações do mundo real) que estão em um banco de dados em algum lugar do mundo.

O servidor não enviará o banco de dados, mas sim, uma representação dos usuários em algum formato. No exemplo que faremos mais à frente, a representação dos usuários será retornada em JSON.

Pois bem, como vamos encontrar esse servidor e obter uma representação do recurso que queremos? Para tanto, vamos utilizar uma Uniform Resource Identifier ou URI, que nada mais é do que uma sequência de caracteres, usados para identificar um recurso de forma global e única. A URI que utilizaremos será a URL http://fiap-on.getsandbox.com/controle/usuarios. Essa URL endereça uma API construída no Sandbox, exclusivamente para exemplificarmos os conceitos que estamos vendo.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Caso não conheça, o Sandbox é uma plataforma web que possibilita o desenvolvimento fácil e rápido de mocks para APIs definidas em Swagger (V2 JSON), API Blueprint, WSDL, RAML ou pela criação de rotas codificadas em JavaScript.

Para mais informações, acesse: https://getsandbox.com/.

Podemos dizer que uma URL (Uniform Resource Locator) é uma URI que, além de identificar unicamente um recurso na web, especifica o protocolo, o servidor, o contexto da aplicação e o meio para acessar a representação do recurso desejado.

Vamos entender como isso acontece esmiuçando a URL:
http://fiap-on.getsandbox.com/controle/usuarios

Protocolo: http

Servidor na rede: fiap-on.getsandbox.com

Contexto da aplicação: /controle

Meio de acesso à representação do recurso: /usuarios

Representação do recurso: No Chrome instale a extensão JSONView (Validate and view JSON documents), digite a URL e você obterá do servidor a representação em JSON do recurso:

Figura 3.1 – Retornando usuários – API fiap-on SandBox
Fonte: Elaborado pelo autor (2017)

Como podemos perceber, uma URL deve ser única por recurso, ou seja, quando queremos receber a lista de usuários, representadas pela notação JSON, devemos sempre utilizar a URL http://fiap-on.getsandbox.com/controle/usuarios

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

E se quisermos acessar apenas o primeiro usuário? A URL deve ser diferente e novamente única, sendo assim, uma implementação simples e intuitiva para obtermos o primeiro usuário, naturalmente seria a URL http://fiap-on.getsandbox.com/controle/usuarios/1 e foi dessa forma que codificamos a API no Sandbox. Testando a URL no Chrome, obtemos:

Figura 3.2 – Retornando usuário de id 1 – API fiap-on SandBox
Fonte: Elaborado pelo autor (2017)

Note que o conceito de URI/URL como identificador único facilita muito o entendimento e o consumo da API. Se você enviar os links para alguma pessoa desinformada sobre a aplicação, creio que ela por si só facilmente concluirá que se trata de um aplicativo web para controle de usuários, que lista um usuário específico ou todos, de acordo com a URL empregada. Concorda?

Para fortalecer esse raciocínio, se a nossa API fosse desenvolvida como um "Big" Web Service utilizando SOAP, nosso usuário teria de enviar um envelope SOAP para realizar a pesquisa. Você se lembra disso? Você conhece algum usuário que sabe fazer sozinho um envelope SOAP?

3.2.1.2 Manipule os recursos utilizando representações

Quando um cliente possui representação de um recurso, ele possui informações suficientes para incluir, modificar ou excluir o recurso no servidor, desde que tenha permissão para fazê-lo. Vamos entender melhor esse conceito mais à frente.

FIAP

Em nossos exemplos, e na maioria dos serviços RESTful, a Notação de Objetos JavaScript ou JSON (JavaScript Object Notation) é largamente utilizada como opção ao XML (eXtensible Markup Language).

3.2.1.3 JSON

Segundo o json.org, JSON é uma formatação leve de troca de dados. Para seres humanos, é fácil de ler e escrever. Para máquinas, é fácil de interpretar e gerar. Está baseado em um subconjunto da linguagem de programação JavaScript, Standard ECMA-262 3ª edição, dezembro/1999. JSON um formato em texto, completamente independente de linguagem, pois usa convenções que são familiares a diversas linguagens.

JSON basicamente é construído em duas estruturas:

- Uma coleção de pares nome / valor;
- Uma lista ordenada de valores;

Como a estrutura do JSON é muito simples, em uma situação que precisamos trocar dados entre uma aplicação cliente e um serviço web, seria muito interessante podermos transformar um objeto em JSON (Marshalling) e enviá-lo para o serviço, seria também muito interessante recebermos um JSON e transformá-lo em um objeto no cliente (Unmarshalling), concorda? Faremos o envio no tópico a seguir, utilizando os verbos HTTP PUT e POST.

Como JSON é independente de plataforma, podemos enviar e receber informações via Web em diversas linguagens e em ambientes de hardware e software totalmente heterogêneos, utilizando Web Services.

Em um cenário hipotético, poderíamos ter aplicações de clientes em hardwares distintos, SOs e linguagens de programação diferentes, transformando objetos JavaScript (Browsers), objetos Java (App) e objetos .Net (Desktop Application) em JSON e enviando para um único serviço, em um Servidor Web, responsável por armazenar em um Banco de Dados ou em um Servidor de Arquivos.

Ainda em nosso cenário hipotético, não seria impossível imaginar os diversos clientes consumindo o mesmo serviço, para receber um JSON formatado a partir de objetos instanciados no Servidor Web com dados do Banco de Dados ou para receber

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

o JSON de um arquivo de texto físico, formatado em JSON, que o Servidor Web busca no Servidor de Arquivos.

Da mesma forma que os clientes podem estar em ambientes de hardware e software diferentes, sem impactar a comunicação, o Web Service executado no Internet Server também pode ser codificado nas mais diversas linguagens, tais como, Java, .NET, PHP, entre outras e pode ser executado em SOs completamente diferentes daqueles dos clientes, sem nada impactar a comunicação entre consumidor e serviço web.

Na essência, JSON é um texto formatado, tanto quanto o XML, portanto, ambos possibilitam o tráfego suave através de firewalls e podem utilizar a porta 80 do protocolo HTTP, possibilitando a chamada de serviços em um Servidor de Internet, sem o bloqueio de um Firewall.

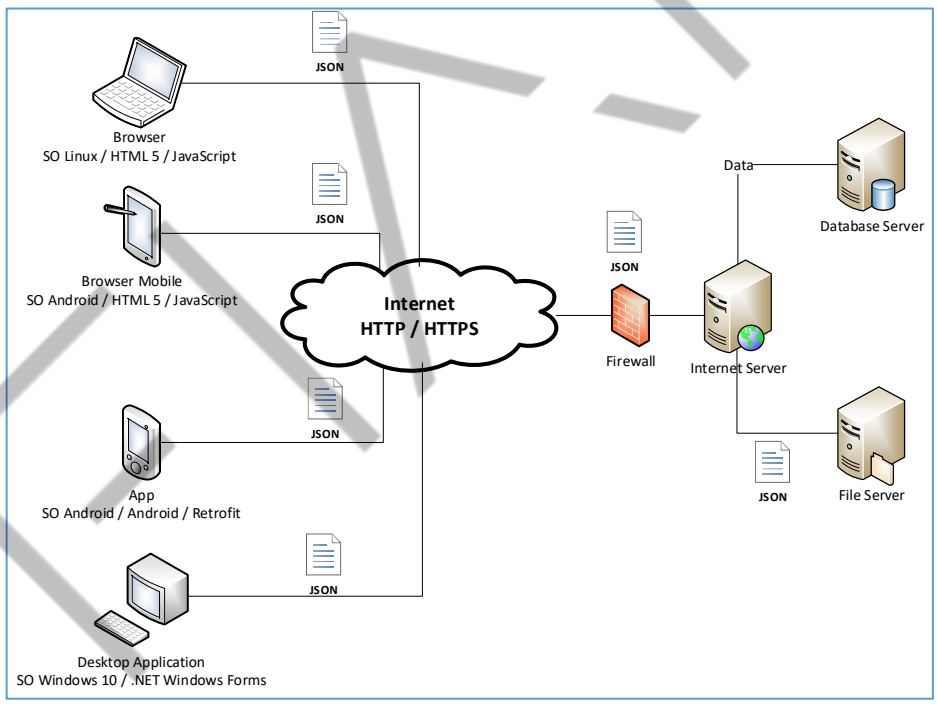

Figura 3.3 – JSON trafegando via HTTP/HTTPs
Fonte: Elaborado pelo autor (2017)

Já vimos no capítulo sobre SOA e “Big” Web Services que os benefícios de interligarmos aplicações em ambientes heterogêneos via web também podem ser atingidos utilizando XML.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3.2.1.4 Por que JSON?

Para responder a essa pergunta, vamos olhar o mesmo recurso: uma lista contendo dois usuários, representado por ambas as formatações.

3.2.1.5 JSON

```
{"usuarios": [
        {"id": 1,
         "nome": "Fernando Gouveia Lima",
         "cpf": "999.999.999-99"},
        {"id": 2,
         "nome": "Fernando Barbosa Lima",
         "cpf": "111.111.111-11"} ]}
```

Código-fonte 3.1 – Lista de usuários representado em JSON
Fonte: Elaborado pelo autor (2017)

3.2.1.6 XML

```
<usuarios>
    <usuario>
        <id>1</id>
        <nome>Fernando Gouveia Lima</nome>
        <cpf>999.999.999-99</cpf>
    </usuario>
    <usuario>
        <id>2</id>
        <nome>Fernando Barbosa Lima</nome>
        <cpf>111.111.111-11</cpf>
    </usuario>
</usuarios>
```

Código-fonte 3.2 – Lista de usuários representado em XML
Fonte: Elaborado pelo autor (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Analisando os dois códigos, podemos perceber facilmente que ambas as representações são autodescritivas e possuem uma hierarquia fácil de entender. Em ambas as representações, as quebras de linha e os espaços não são obrigatórios, foram empregados apenas para facilitar a visualização.

Da mesma forma, não é difícil percebermos que a representação JSON emprega array e não usa tags de fechamento para delimitar um dado de outro. Portanto, tem um tamanho menor do que uma representação em XML e, na maioria dos casos, é mais rápida de se escrever e ler.

```
Array  "usuarios": []   X     Tag <usuarios></usuarios> End Tag


name/value    "id": 1        X     Tag <id> Value 1 </id> End Tag
```

Código-fonte 3.3 – Comparações entre Arrays declarados em JSON e XML
Fonte: Elaborada pelo autor (2017)

Ainda podemos acrescentar mais uma diferença, o JSON não tem a obrigatoriedade de ter um elemento raiz como o XML, ou seja, o name "usuarios": é opcional, enquanto as TAGs <usuarios></usuarios> que determinam o elemento raiz do XML são obrigatórias. Apesar de menos intuitivo, muitas aplicações processam o JSON para representar o recurso lista de usuários, estruturando-o da seguinte forma:

```
{ [
    {"id": 1,
     "nome": "Fernando Gouveia Lima",
     "cpf": "999.999.999-99"},
    {"id": 2,
     "nome": "Fernando Barbosa Lima",
     "cpf": "111.111.111-11"} ]}
```

Código-fonte 3.4 – Lista de usuários em JSON sem o array usuários
Fonte: Elaborado pelo autor (2017)

Dando dois passos para trás, enfatizamos que “na maioria dos casos, JSON é mais rápido de ler e escrever” e não que “em todos os casos”, pois a linguagem de programação, a escolha das bibliotecas e da técnica de análise (parsing) utilizada, impactam diretamente na velocidade de escrita e leitura em ambas as formas de representação. Em nossos estudos, empregaremos Java como a linguagem de

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

programação e faremos Marshalling e Unmarshalling, utilizando bibliotecas desenvolvidas para Java.

De qualquer maneira, podemos afirmar que aplicações aderentes às restrições impostas pelo estilo REST e que utilizam JSON como representação, se saem muito bem em situações em que há limitação de recursos ou quando um baixo consumo de largura de banda é necessário. Ou seja, JSON é muito empregado em soluções, nas quais o tamanho do fluxo de dados entre o cliente e o servidor é de grande importância. JSON é utilizado por grandes empresas que enfrentam desafios com essas características, tais como, Google, Facebook, Twitter, entre outras.

Para mais informações sobre JSON, acesse:
http://www.json.org e https://www.w3schools.com/js/js_json_intro.asp

Apesar da especial atenção que demos a JSON e XML, precisamos compreender que um recurso pode ter várias representações, ou seja, podemos desenvolver aplicações que retornem representações diferentes de acordo com cenários ou necessidades específicas de cada cliente. Como exemplo, se o cliente final for uma aplicação Mainframe, nada nos impede de retornar uma representação em CSV em vez de JSON ou XML.

3.2.1.7 Manipule os recursos por meio de operações padronizadas

Quando falamos sobre manipular os recursos de forma padronizada, falamos sobre enviar e receber informações via rede de forma organizada e unificada. Podemos afirmar que esse é o principal papel do protocolo HTTP, também podemos dizer que aplicações RESTful devem se aproveitar das características desse protocolo, bem como, dos demais padrões Web, para aderir às restrições REST.

3.2.1.8 HTTP Headers

Em uma comunicação comum utilizando HTTP, o cliente sempre solicita algo e então, o servidor responde. Sendo assim, a responsabilidade do uso correto do protocolo é tanto do cliente quanto do servidor.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Mensagens HTTP trocadas entre eles são formadas obrigatoriamente por uma linha inicial, conhecida como Request Line (em solicitações do cliente) e reconhecida como Status Line (em respostas do servidor), depois por um conjunto de Header Fields (Request or Response) e, opcionalmente, por um Body para envio ou retorno de conteúdo.

Como vimos nas requisições que retornaram uma lista de usuários e depois um usuário específico, fizemos ambas as solicitações sem enviar nada no Body, já que as operações não precisavam. Você se lembra?

Uma linha de solicitação ou Request Line, contém o verbo HTTP, o URL do recurso e a versão do protocolo HTTP que está sendo usado, já o os campos do cabeçalho, contêm metadados descritos em pares (nome e valor), cada campo contêm um nome e um valor, com uma informação específica sobre a solicitação.

Os clientes usam a primeira linha e os campos do cabeçalho de uma mensagem de solicitação para processar um recurso, para se identificar e para informar como a representação do recurso deve ser retornada pelo Servidor.

Como exemplo, vamos dar uma olhada na mensagem de solicitação HTTP que fizemos no Chrome para http://fiap-on.getsandbox.com/controle/usuarios/1

```
GET /controle/usuarios/1 HTTP/1.1        } Linha de Solicitação
Host: fiap-on.getsandbox.com
Accept: */*
Accept-Language: pt-BR,pt;               } Cabeçalho
User-Agent: Chrome/61.0.3163.100
Accept-Encoding: gif, deflate
```

Código-fonte 3.5 – Mensagem de requisição HTTP
Fonte: Elaborado pelo autor (2017)

Nessa mensagem, podemos ver que uma solicitação com o método **GET** foi feita para o servidor **fiap-on.getsandbox.com** na URL **/controle/usuarios/1** aceitando qualquer tipo de representação retorno ***/*** na língua **pt-BR,pt.** A solicitação foi feita do navegador **Chrome/61.0.3163.100** que está apto para

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

processar o conteúdo de resposta comprimido, usando os algoritmos **`gzip, deflate.`**

E o Body?

Nessa mensagem de solicitação, ele não foi necessário, mas em outras mensagens, será. Em solicitações PUT (Update) e POST (Create) não teremos de enviar para o Servidor uma representação do recurso que queremos atualizar ou criar por meio dele?

Nesses casos, o Body conterá as informações que precisamos enviar para o Servidor e ele as processará de acordo com o verbo HTTP e o URL que estiverem na Request Line da mensagem.

Vimos uma mensagem de requisição GET. Vamos olhar a resposta?

```
HTTP/1.1 200 OK                                  } Linha de Solicitação
Server: nginx
Date: Mon, 30 Oct 2017 15:36:31 GMT
Content-Type: application/json
Content-Encoding: gzip
```

Código-fonte 3.6 – Mensagem de resposta HTTP
Fonte: Elaborado pelo autor (2017)

A primeira linha de uma mensagem de resposta do protocolo HTTP é conhecida como Status Line e é consistida pela versão de protocolo, seguida por um código de status numérico e uma frase explicativa sobre o código. O serviço fiap-on utilizou-a para informar o sucesso do processamento do método GET, por meio do Status Code 200.

Já os campos do cabeçalho informam que o servidor web que processou o pedido é um Nginx, a data e o horário do processamento no servidor, o tipo do conteúdo retornado no body e como ele foi comprimido. Aqui está o Body da nossa resposta em JSON. Lembra-se?

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Figura 3.4 – Retornando usuário de id 1 – API fiap-on SandBox
Fonte: Elaborado pelo autor (2017)

Como cada requisição de um cliente feita a um serviço está condicionada a um verbo HTTP e como comentamos que os serviços RESTful agem de forma diferente de acordo com eles, vamos entender mais um pouco sobre os mais utilizados em APIs REST.

3.2.1.9 Media Types

Os protocolos da Internet foram cuidadosamente projetados para serem facilmente extensíveis em determinadas áreas. Muitos protocolos, incluindo o HTTP, são capazes de transportar conteúdos rotulados. O mecanismo utilizado para rotular esses conteúdos, são os Media Types.

Os tipos mais comuns de Media Types são text, application, image, vídeo, audio e vnd. Os Media Types são divididos em tipos / subtipos e acrescidos de parâmetros quando uma informação adicional é necessária. Um exemplo seria “text/html;charset=utf-8”

Para exemplificarmos, quando fizemos uma solicitação para http://fiap-on.getsandbox.com/controle/usuarios/1, informamos a nossa API, via cabeçalho HTTP Accept, que esperávamos qualquer tipo de conteúdo de retorno, ou seja, media type */*.

Na resposta, a API retornou no cabeçalho Content-Type o Media Type application/json, para que o cliente saiba que o retorno deve ser processado como JSON. Caso tivéssemos solicitado via Accept um conteúdo em XML e a API não pudesse retorná-lo, o Status Code 415 deveria ser retornado pela API. APIs RESTful utilizam vários tipos de Media Types, mas os mais comumente utilizados são application/json e application/xml.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Entenderemos mais à frente sobre HTTP Status Code e faremos exemplos com Media Type diferentes.

3.2.1.10 HTTP Methods

3.2.1.10.1 GET

Em nossos dois exemplos de pesquisa sobre informações de usuários, utilizamos uma das operações padronizadas pelo HTTP, a operação GET.

Para confirmar essa afirmação, pressione F12 no Chrome, busque novamente por http://fiap-on.getsandbox.com/controle/usuarios, selecione Network e depois usuários, você verá em Headers que o método de requisição foi o GET e o código de sucesso foi o 200, conforme imagem abaixo.

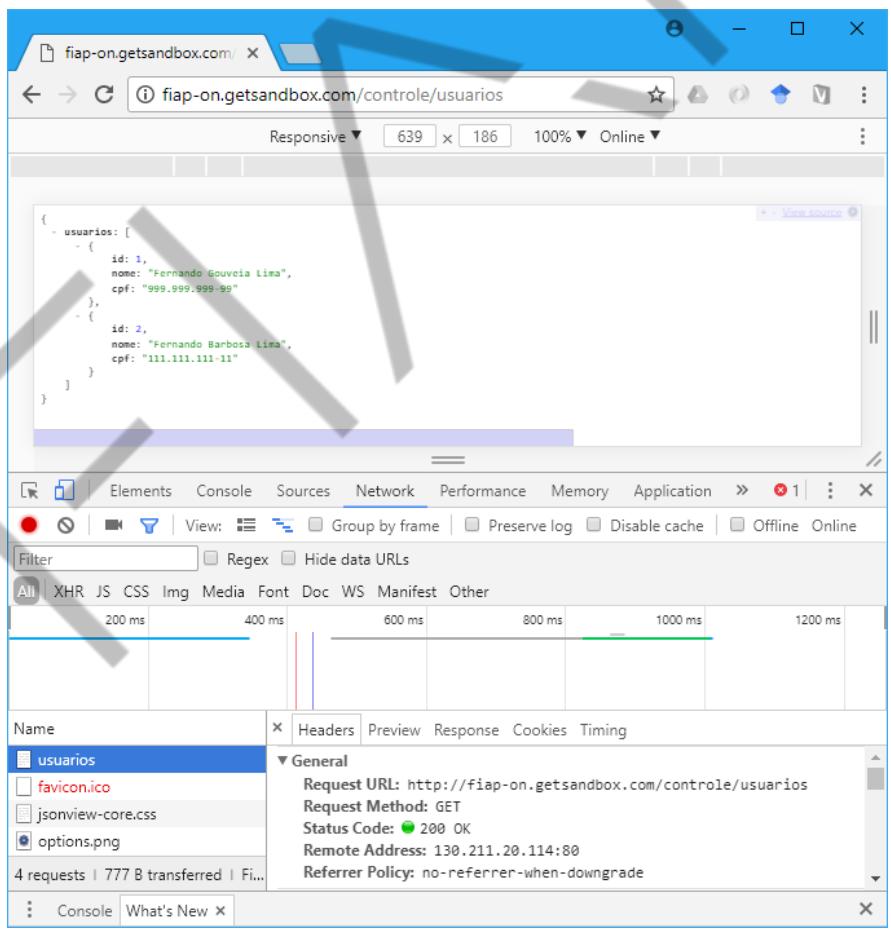

Figura 3.5 – Visualizando HTTP Headers no Chrome - API fiap-on SandBox
Fonte: Elaborado pelo autor (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Como comentamos, o protocolo HTTP nos fornece uma interface de operações padronizadas. Essa interface é muito utilizada por aplicações RESTful para operações CRUD.

Além do verbo HTTP GET para leitura (R), aplicações RESTful comumente utilizam os verbos POST para criação (C), PUT para atualização (U) e DELETE para excluir (D) um determinado recurso via rede. O protocolo HTTP possui mais verbos, além dos quatro já citados, mas neste momento vamos focar neles.

Você se lembra quando mencionamos que se o cliente possui permissão e uma representação do recurso, ele pode incluir, modificar ou excluir esse recurso no servidor?

Uma aplicação RESTful se vale das informações contidas em uma mensagem de solicitação HTTP para identificar e realizar a operação que o cliente deseja realizar. Os exemplos a seguir possuem como premissa a instalação do Postman, uma ferramenta utilizada por desenvolvedores e analista de testes, para auxílio na criação, testes e documentação de APIs.

Para instalar a ferramenta de acordo com a sua plataforma, baixe da seguinte URL https://www.getpostman.com. Após os detalhes de criação de conta superados, ao executar o Postman, selecione Collection e siga os demais passos.

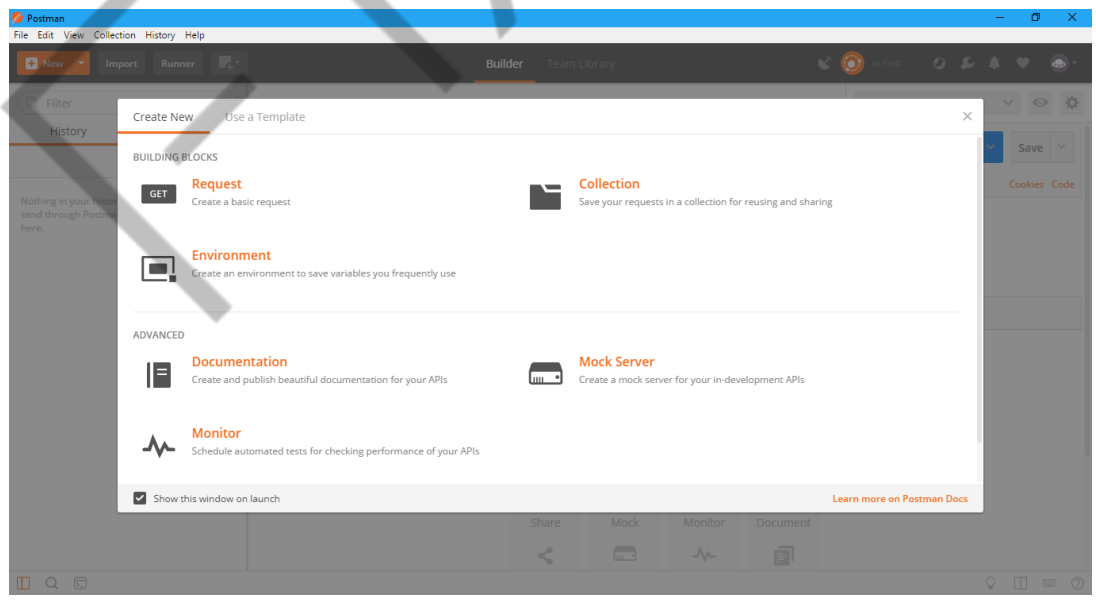

Figura 3.6– Criando uma Collection no Postman
Fonte: Elaborado pelo autor (2017)

Nomeie a Collection como FIAP Usuários.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

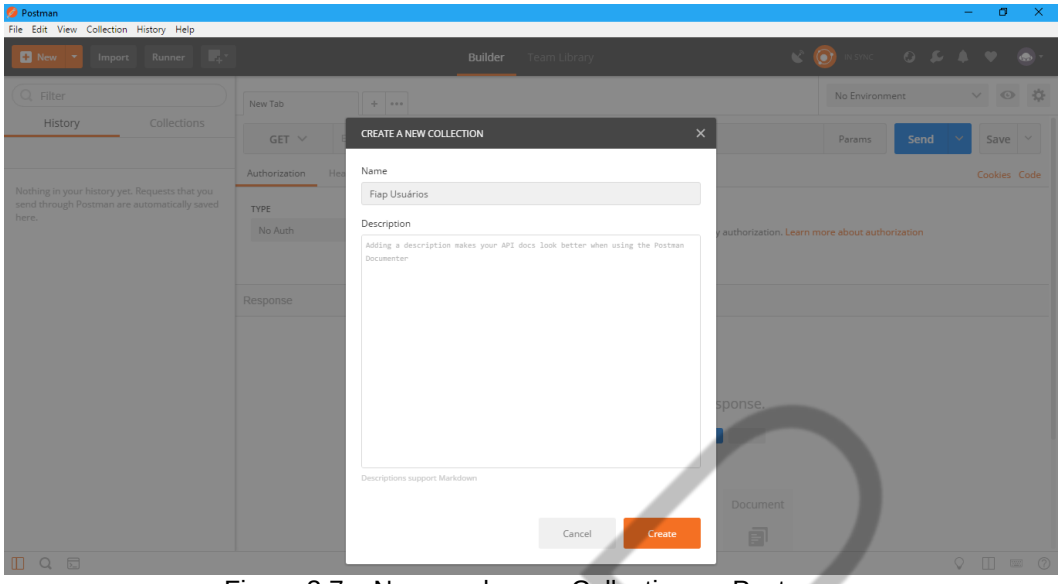

Figura 3.7 – Nomeando uma Collection no Postman
Fonte: Elaborado pelo autor (2017)

Crie o primeiro método da Coleção, utilizando o método GET digite a URL http://fiap-on.getsandbox.com/controle/usuarios e clique em Send, nossa API retornará a lista de usuários em JSON, conforme imagem abaixo.

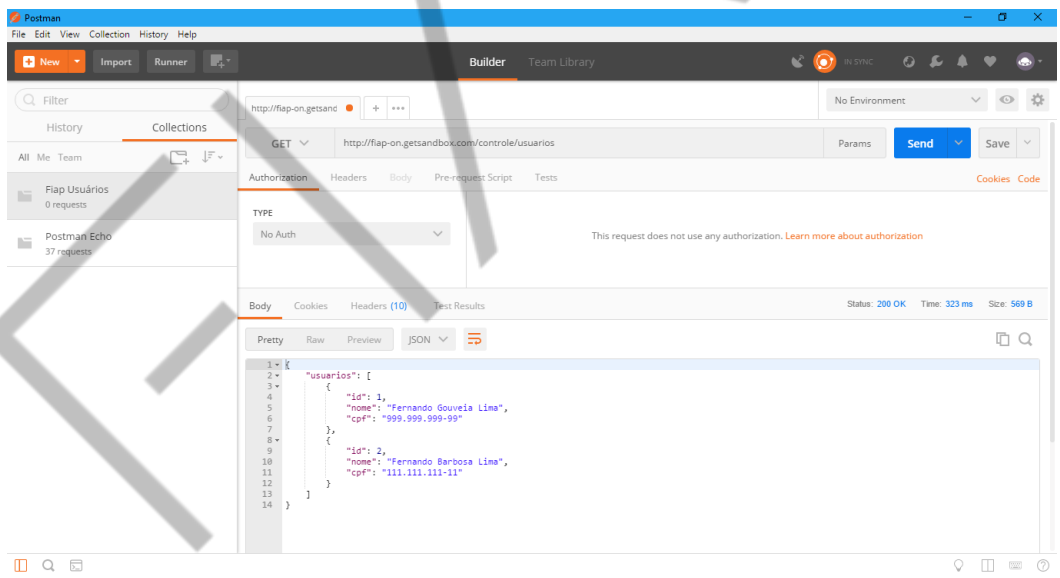

Figura 3.8 – Criando uma requisição GET no Postman
Fonte: Elaborado pelo autor (2017)

Explore a aba Headers, veja o Status 200 OK e o tempo de resposta da API. Na sequência, clique em Save e salve essa requisição na coleção FIAP Usuários, conforme imagem abaixo.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

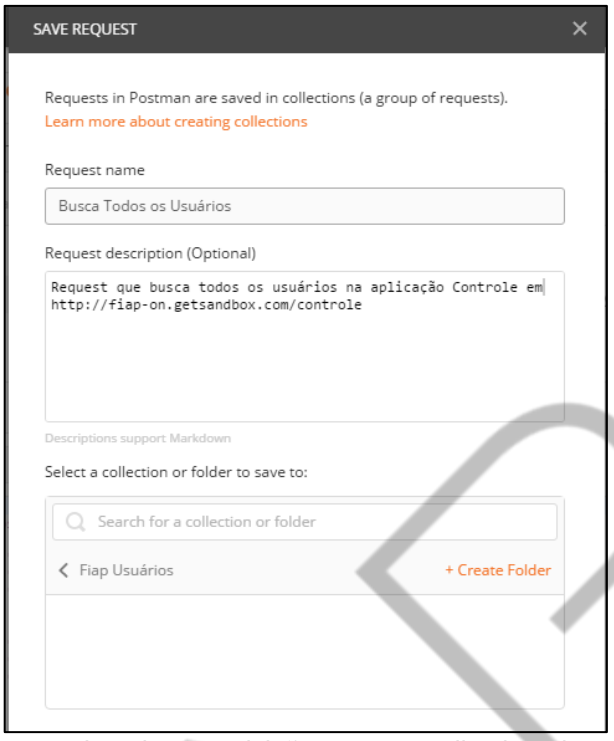

Figura 3.9 – Salvando a requisição GET na Collection Fiap Usuários
Fonte: Elaborado pelo autor (2017)

Fazendo isso, você deixa a pesquisa que realizamos gravada na ferramenta, de forma identificada, documentada e separada por coleções, geralmente ligadas a uma aplicação ou a um módulo de uma aplicação maior que se pretende testar.

Caso queira, repita essas ações para a URL que retorna apenas um usuário. No momento, este não é o nosso foco, vamos mirar nos demais métodos CRUD da nossa API, utilizando os verbos HTTP PUT, POST e DELETE. Apesar de não ser a melhor prática, vamos executar essas três operações e depois realizar uma nova consulta com GET para aferir os resultados.

3.2.1.11 PUT

Primeiro, vamos atualizar o usuário 1 utilizando o Postman. Pois bem, para realizar essa operação, precisamos enviar a representação do recurso que queremos alterar para a aplicação RESTful, utilizando o verbo PUT.

Vamos lá, no Postman adicione uma nova aba, utilize a URL da imagem e altere o método de GET para PUT. Em Headers, adicione Key e Value, conforme imagem abaixo, pois precisamos adicionar ao cabeçalho a informação de que enviaremos em nossa requisição o conteúdo do Body em JSON.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

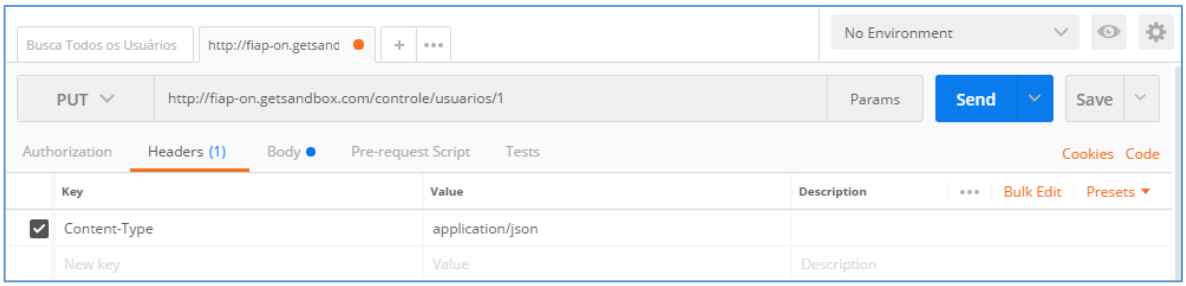

Figura 3.10 – Criando uma requisição PUT no Postman
Fonte: Elaborado pelo autor (2017)

Em Body, selecione raw e escreva a representação do recurso que queremos alterar. Clique em Send e você obterá o status OK, conforme imagem abaixo.

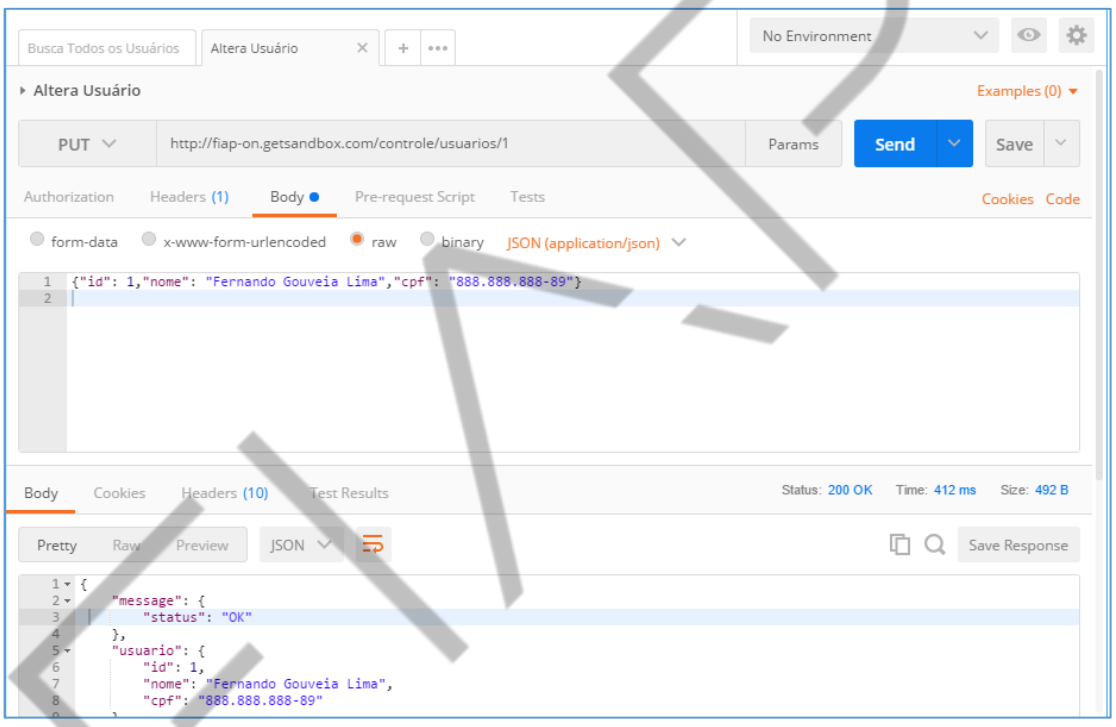

Figura 3.11 – Enviando a requisição PUT no Postman
Fonte: Elaborado pelo autor (2017)

A operação obteve sucesso, retornou o Status Code 200 OK e como parte da mensagem de retorno, o recurso que foi atualizado para evitar que o consumidor do serviço tenha que fazer uma nova solicitação via GET para obter a representação atualizada do usuário de id = 1. Essa é uma boa prática de desenvolvimento empregada pelo REST, tanto em alteração de recursos (PUT), quanto em criação (POST). Salve essa operação na Collection Fiap Usuários com o nome que desejar.

O método PUT é utilizado para alterar um recurso enviando a sua representação completa. Caso você queira implementar uma atualização enviando uma representação parcial, estude a abordagem utilizando o verbo PATCH.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

A RFC 7231 orienta que o método PUT altere o recurso, quando encontrado. Quando não encontrado, crie o recurso. Em REST, desconheço a obrigatoriedade de implementarmos uma API CRUD dessa forma. Sugiro que isso fique a critério do design da solução e seja documentado.

As restrições que considero sobre esse assunto são:

- Se um recurso for atualizado com PUT, retorne 200 OK quando retorná-lo no body, 204 No Content, se atualizá-lo, mas nada for retornado no body
- Se um recurso for criado com PUT, retorne o Status Code 201 Created, a URI do novo recurso, no cabeçalho Location e uma representação do novo recurso criado no body.

O mock FIAP utilizou o PUT apenas para alterações de recursos.

3.2.1.12 POST

Para criar um recurso por meio da nossa API, adicione uma nova aba no Postman, utilize a URL da imagem abaixo e altere o método para POST. Em Headers, adicione Key e Value, conforme fizemos no método PUT. Em Body, selecione raw e escreva a representação do recurso que desejamos criar. Clique em Send e você obterá o status OK, conforme imagem abaixo.

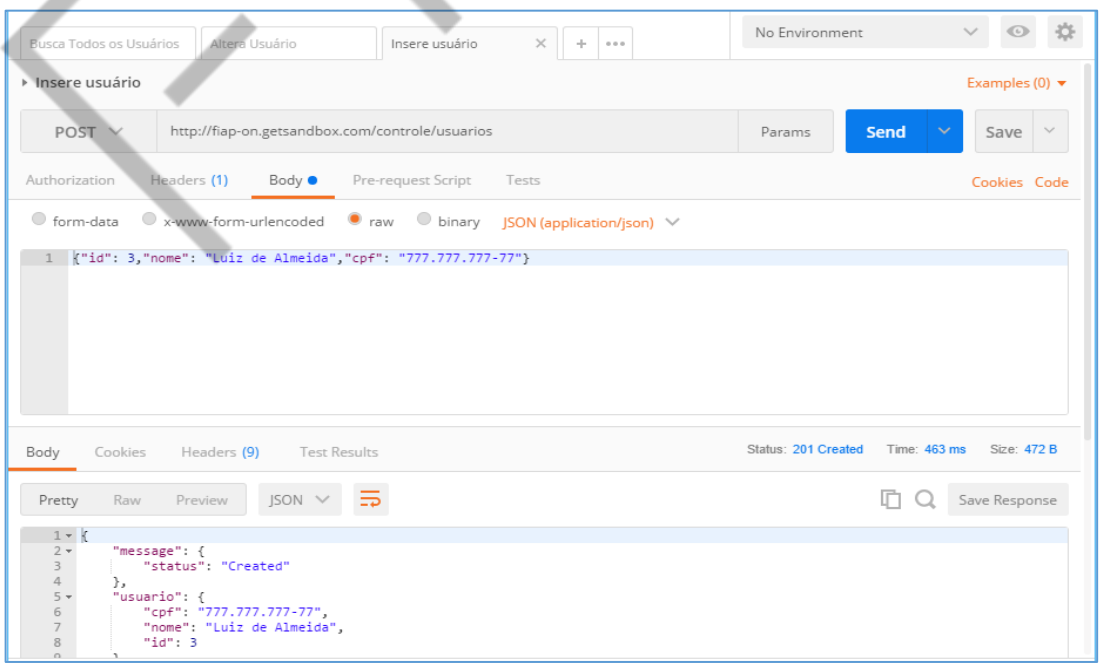

Figura 3.12 – Criando e enviando uma requisição POST no Postman
Fonte: Elaborado pelo autor (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Salve a operação na Collection. Note que para criar um usuário, utilizamos a mesma URL que retorna todos os usuários, mas desta vez, empregamos o método POST em vez do GET e enviamos uma representação do usuário para criação. Simples, não?

Note que a API retornou o usuário criado e utilizou o Status Code 201 – Created para informar o sucesso da operação e não mais o Status Code 200 – OK. Ainda temos mais um uso relevante do protocolo HTTP, a API utilizou o cabeçalho Location, para retornar a URI completa do novo recurso.

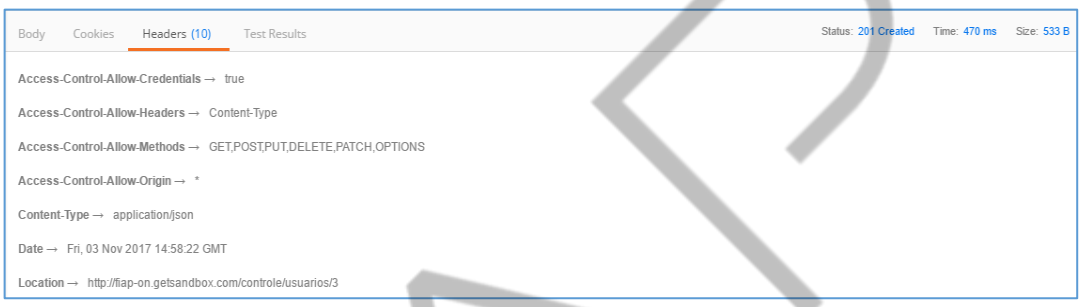

Figura 3.13 – Headers HTTP e Status Code da resposta ao POST no Postman
Fonte: Elaborado pelo autor (2017)

Creio que neste ponto, ficou mais fácil perceber como a nossa aplicação RESTful aproveita as características do HTTP em seu benefício.

3.2.1.13 Delete

Agora, vamos excluir um usuário por meio da nossa API. Para tanto, adicione outra aba no Postman, utilize a URL da imagem abaixo e altere o método para DELETE. Clique em Send e você obterá o status OK, conforme imagem abaixo.

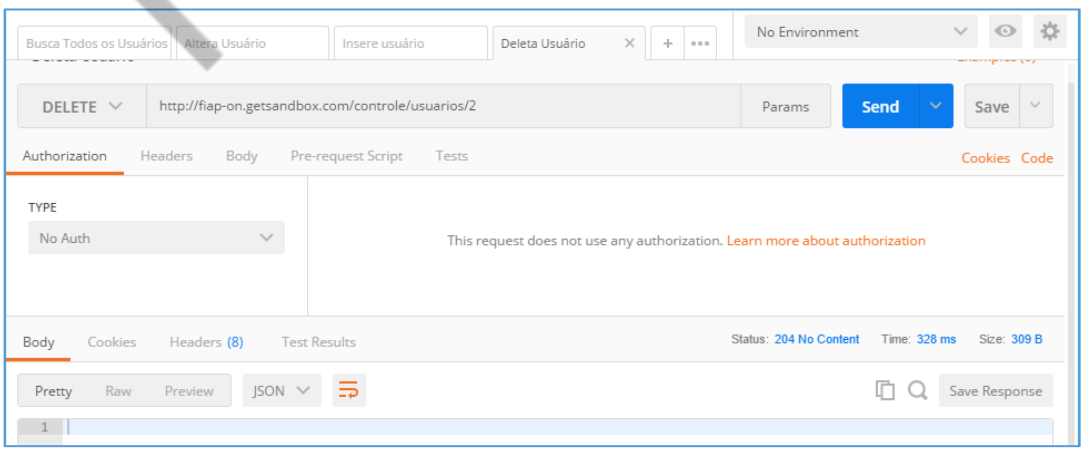

Figura 3.14 – Criando e enviando uma requisição DELETE no Postman
Fonte: Elaborado pelo autor (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Importante comentar que o DELETE pode disparar uma ação na API que exclui o recurso ou apenas o marca para não ser mais disponibilizado via GET. Isso dependerá da regra de negócio e da forma como o back-end está desenvolvido.

Como resultado, receberemos da API o Status Code 204, indicando que o servidor cumpriu com êxito a solicitação e que não há conteúdo adicional no corpo da resposta.

Se fizermos uma busca ao usuário excluído em http://fiap-on.getsandbox.com/controle/usuarios/2, teremos como resposta o Status Code 404 Not Found e uma mensagem tratada pela API, conforme imagem abaixo.

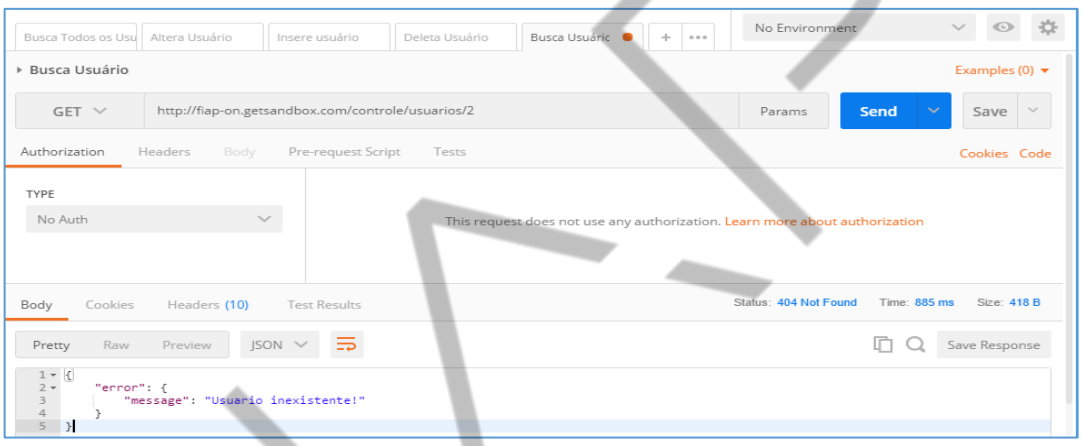

Figura 3.15 – Criando e enviando uma requisição de teste do DELETE no Postman
Fonte: Elaborado pelo autor (2017)

Por fim, vamos testar se as operações Update (PUT), Create (POST) e Delete (DELETE) deram certo. Volte na aba Busca Todos os Usuários e clique em Send.

Utilizando GET, você obterá um JSON com o usuário de id 3 criado e sem o usuário 2, que excluímos no passo anterior.

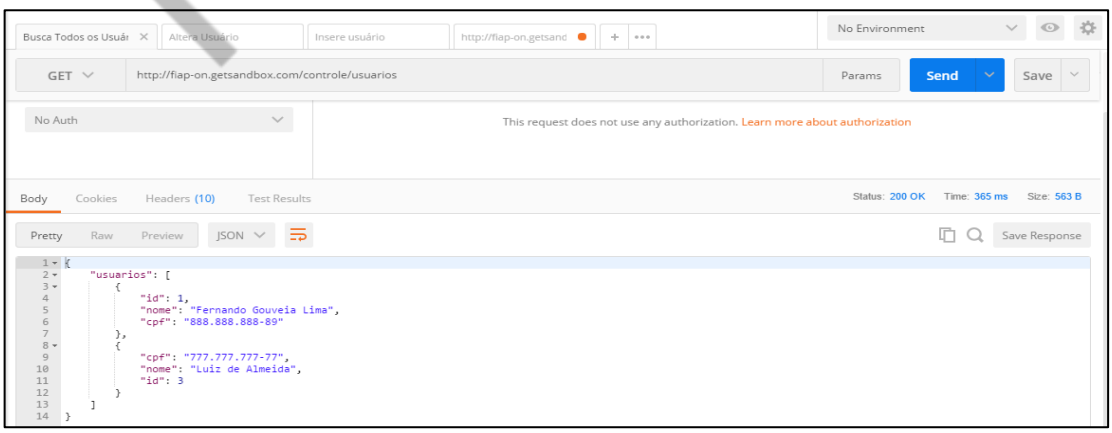

Figura 3.16 – Enviando uma requisição GET de teste no Postman
Fonte: Elaborado pelo autor (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Novamente, recebemos o Status Code 200 OK, informando que a requisição foi processada com sucesso e que há informação contida no Body da resposta.

3.2.1.14 Respostas padronizadas

Como podemos perceber, em todas as respostas da nossa API, nós recebemos um código de status. Nas requisições PUT (Update) e POST (Create), nós incluímos no cabeçalho HTTP da solicitação uma informação importante, o formato da representação contida no Body. Em nossos exemplos, utilizamos o campo Content-Type, com o valor application/json para informar à API que enviamos uma representação JSON para criar ou alterar um recurso.

Dessa forma, o cliente e o serviço utilizaram propriedades do protocolo HTTP para trocar informações adicionais ao conteúdo das mensagens e, se analisarmos bem, informações muito relevantes.

Vamos entender um pouco mais sobre esse contexto e descobrir como o estilo REST novamente se beneficia da arquitetura Web e do protocolo HTTP.

3.2.1.15 HTTP Status Code

Como toda a requisição enviada a um Servidor Web via HTTP retorna um código de status, os serviços RESTful utilizam parte da mensagem de resposta do protocolo para informar ao cliente sobre o resultado da solicitação que ele fez.

O HTTP define dezenas de códigos de status que podem ser usados para retornarmos tal resultado. Face à quantidade, os Status Code estão divididos em cinco categorias, apresentadas a seguir.

CATEGORIA	DESCRIÇÃO
1xx: informativo	Indica uma resposta provisória, de solicitação recebida, dando continuidade ao processo.
2xx: Sucesso	Indica que a ação solicitada pelo cliente foi recebida, compreendida, aceita e processada com êxito.
3xx: Redirecionamento	Indica que o cliente deve tomar alguma ação adicional para completar seu pedido.
4xx: Erro do cliente	Retorna códigos para os casos em que o cliente pode ter cometido um erro.
5xx: Erro do servidor	Indica que o servidor assumiu um erro ao processar a solicitação.

Quadro 3.1 – Categorias de códigos de status para o protocolo HTTP
Fonte: Elaborado pelo autor (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Conhecer os códigos de status HTTP e empregá-los corretamente, de acordo com o status do processamento, é uma responsabilidade importante que cabe aos desenvolvedores de APIs RESTful. Sendo assim, vamos conhecer alguns dos principais códigos e dos empregos mais comuns, baseados na RFC7231 https://tools.ietf.org/html/rfc7231.

3.2.1.16 Uso Geral

- 301 Moved Permanently: Significa que o recurso solicitado foi realocado permanentemente. A resposta deve conter um cabeçalho Location com a nova URL.
- 303 See Other: A requisição foi processada, mas a resposta ao pedido pode ser encontrada em URI diferente e deve ser recuperada usando um método GET nesse recurso. O cabeçalho Location informa onde a resposta do processamento está.
- 307 Temporary Redirect: Semelhante ao 301, só que indicando que o redirecionamento é temporário.
- 400 Bad Request: O pedido não pôde ser entendido pelo servidor devido à sintaxe com problemas.
- 401 Unauthorized: O pedido requer autenticação de usuário, as credenciais de autenticação estão faltando, ou, se fornecidas, foram recusadas para acessar o recurso.
- 403 Forbidden: O servidor entendeu o pedido, mas está se recusando a cumpri-lo. A autorização não ajudará e o pedido não deve ser repetido.
- 404 Not Found: O servidor não encontrou nada que corresponda ao Request-URI.
- 405 Method Not Allowed: O método especificado não é permitido para o recurso identificado.
- 409 Conflict: O pedido não pôde ser concluído devido a um conflito com o estado atual do recurso. Este código só é permitido em situações em que se espera que o usuário possa resolver o conflito e reenviar a

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

solicitação. Um exemplo comum é a criação de um recurso que deveria ser único, mas entra em conflito com um recurso semelhante, já existente.

- 410 Gone: O recurso já não está disponível e que é provável que a condição seja permanente.
- 415 Unsupported MediaType: Indica que o servidor se recusa a atender o pedido, porque a solicitação está em um formato não suportado pelo método no recurso. Exemplo, solicitação de um JSON e o retorno é um recurso PDF.
- 500 Internal Server Error: Indica que o servidor encontrou uma condição inesperada que impediu o cumprimento o pedido.
- 503 Service Unavailable: Indica que o servidor atualmente não pode lidar com o pedido devido a uma sobrecarga temporária ou manutenção programada, o que provavelmente será atenuado após algum tempo.
- 504 Gateway Timeout: Indica que o servidor, enquanto atuava como um gateway ou proxy, não recebeu uma resposta a tempo de um servidor que precisava acessar para completar o pedido ou a solicitação não pode ser atendida, devido a alguma outra falha que causou timeout.

3.2.1.17 GET

- 200 OK: O pedido foi bem-sucedido e a resposta contém a representação solicitada.
- 302 Found: Indica que o recurso de destino reside temporariamente em um URI diferente e funciona como uma resposta de redirecionamento em que a URI temporária pode ser obtida no cabeçalho da resposta.
- 304 Not Modified: Indica que uma solicitação GET condicional foi recebida e resultaria em uma resposta 200 (OK) se não fosse pelo fato de não ser necessário retorno, pois o cliente já possui uma representação válida do recurso solicitado.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3.2.1.18 POST ou PUT

- 201 OK: Indica que o pedido foi cumprido e resultou em um ou mais novos recursos criados. Tipicamente a identificação do recurso criado e uma representação dele são retornados.

3.2.1.19 PUT

- 200 OK: O pedido foi bem-sucedido e o corpo da resposta contém a representação atualizada.

3.2.1.20 DELETE

- 204 No Content: Indica que o servidor cumpriu com êxito a solicitação e que não há conteúdo adicional no corpo da resposta.

3.2.1.21 POST, PUT e DELETE

- 202 Accept: O pedido foi aceito para processamento, mas o processamento não foi concluído. Não há garantias sobre o resultado do processamento posterior. O propósito deste código é permitir ao servidor receber e confirmar solicitações para outros processamentos. Não há funcionalidades no HTTP para reenviar um status de forma assíncrona quando o processamento posterior finalizar.

3.2.1.22 Use mensagens autodescritivas

Cada mensagem de uma aplicação RESTful deve incluir informações suficientes para descrever como deve ser processada. Os recursos são desacoplados de sua representação para que seu conteúdo possa ser acessado em vários formatos, como o JSON que vimos em nossos exemplos, ou XML, HTML, TXT, entre outros Media Types.

Metadados sobre o recurso estão disponíveis e devem ser usados, por exemplo, para a aplicação informar erros ou sucessos de operações (Status Code),

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

informar o formato de representação apropriado (Key e Value) e executar autenticação ou controle de acesso (Authorization) que veremos mais à frente.

3.2.1.23 Utilize HATEOAS

Como vimos, os clientes fornecem estado por meio do conteúdo do Body enviado, dos parâmetros de consulta, dos cabeçalhos de solicitação e da URL solicitada. Os serviços fornecem estado aos clientes por meio do conteúdo do Body, dos Status Codes de resposta e dos cabeçalhos de resposta. Isso é tecnicamente referido como hipermídia (ou hiperlinks no hipertexto).

HATEOAS (Hypermedia as the Engine of Application State) considera que, quando necessário, os links para recuperação do próprio recurso ou de recursos relacionados devem estar contidos no Body ou no Head retornado. Como exemplo, utilizando HATEOAS, poderíamos retornar em nossa API o departamento que cada usuário trabalha no Body do response, como um link para o recurso departamento da seguinte forma:

```
{ [
    {"id": 1,
     "nome": "Fernando Gouveia Lima",
     "cpf": "999.999.999-99",
     "links": [{
      "rel": "departamento",
      "href": "http://fiap-on.getsandbox.com/controle/departamentos/1"
       } ]
    },
    {"id": 2,
      "nome": "Fernando Barbosa Lima",
      "cpf": "111.111.111-11",
      "links": [{
       "rel": "departamento",
       "href":"http://fiap-on.getsandbox.com/controle/departamentos/3"
        } ]}
 ] }
```

Código-fonte 3.7 – Exemplo de resposta JSON com HATEOAS
Fonte: Elaborado pelo autor (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Em que “rel” significa e descreve o relacionamento. Já “href” é um URL completo que aponta exclusivamente um recurso relacionado, com informações complementares ao recurso usuario. Esse foi um exemplo ilustrativo, o retorno de departamento em /controle/departamentos/ não está implementado.

3.2.2 Não mantenha o estado - Stateless

Com certeza, já nos deparamos com o desenvolvimento de soluções que utilizam um contêiner web para nos fornecer variáveis de "sessão", capazes de manter o estado de dados trocados entre o cliente e a aplicação web dinâmica. Apesar de útil na arquitetura de páginas dinâmicas, essa não é uma característica do estilo REST.

Como REST é um acrônimo para a Transferência de Estado Representacional, trocar mensagens sem manter o estado no servidor é uma restrição do estilo. Essencialmente, em REST, o estado necessário para lidar com a solicitação deve estar contido dentro da própria solicitação, seja como parte do URL, dos parâmetros enviados, do body ou cabeçalhos HTTP da mensagem.

A URL identifica um recurso no servidor de forma exclusiva e o body contém o estado ou a mudança de estado desse recurso. Então, o retorno se dá ao cliente por cabeçalhos, status code e body da resposta. Após o processamento, nenhuma identificação ou informação sobre as mensagens trocadas fica armazenada em variáveis de memória no servidor.

Comunicar sem estado, permite maior escalabilidade, pois o servidor não precisa manter, atualizar ou comunicar nenhum tipo de estado do cliente. Além disso, comunicações consecutivas tornam-se independentes do servidor que as atende e os balanceadores de carga da rede não precisam se preocupar com configurações adicionais, para controlar sessões.

3.2.3 Utilize Cache

Como na World Wide Web, os clientes podem armazenar respostas em cache. As respostas devem implícitas ou explicitamente se definir como cacheáveis ou não, para impedir que os clientes reutilizem dados obsoletos ou inapropriados, em resposta a pedidos adicionais. Se bem gerenciado, o armazenamento em cache reduz ou

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

elimina algumas interações cliente-servidor, melhorando ainda mais a escalabilidade e o desempenho de soluções REST.

3.2.4 Utilize arquitetura Client-Server

Como podemos perceber, a interface uniforme separa os clientes dos servidores. Tal separação, permite que os clientes não fiquem preocupados com o processamento e com a persistência de dados, que são responsabilidades dos servidores, possibilitando que a portabilidade do código cliente seja evoluída livremente.

Por sua vez, os servidores não se preocupam com a interface do usuário ou com a sessão do usuário, de modo que os servidores possam ser simples, performáticos e escaláveis.

Em nossos exemplos, para o cliente Postman pouco importa se a API RESTful foi desenvolvida em Java, .NET ou JavaScript. Da mesma forma, nossa API JavaScript pouco se importa se a requisição vem de uma aplicação cliente como o Postman ou de um Browser.

Nessa arquitetura, servidores e clientes podem ser desenvolvidos, mantidos e substituídos livremente, desde que a maneira como trocam as mensagens não seja alterada. Perceba que em certo ponto, voltamos a comentar sobre Stateless.

Como exemplo, se um servidor que acabou de responder a um cliente for desligado para manutenção, como cada mensagem de solicitação deve conter o estado, outro servidor pode assumir uma solicitação subsequente e dar andamento ao atendimento ao cliente, sem impacto algum para ambos. Da mesma forma, se incluirmos um novo servidor de aplicação na Farm com a nossa API, ele estará apto a atender a mesma solicitação subsequente, pois não depende de históricos mantidos em sessão.

Importante para escalabilidade, não?

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3.2.5 Arquitete o sistema em camadas

Um cliente que utilizar um sistema em camadas normalmente não pode dizer se ele está trocando mensagens com o servidor final ou com um servidor intermediário ao longo do caminho. Servidores intermediários podem melhorar a escalabilidade do sistema, possibilitando o balanceamento de carga e fornecendo caches. Além disso, camadas adicionais também podem ser utilizadas para impor políticas de segurança à solução.

3.2.6 Forneça código sobre demanda

Segundo Fielding, essa restrição REST permite que a capacidade funcional do cliente seja ampliada, baixando e executando código na forma de applets ou scripts. Ainda segundo ele, permitir que os recursos sejam baixados após a implantação melhora a extensibilidade do sistema, mas também reduz a visibilidade e, portanto, é uma restrição opcional dentro do estilo REST.

3.2.7 Aplicações RESTful

Para ser considerada como RESTful, a aplicação deve estar aderente às cinco restrições obrigatórias do estilo REST que comentamos: Uniform Interface, Stateless, Cacheable, Client-Server e Layered System. A única considerada como opcional é a Code on Demand.

Claro que as características de hardware, SO e linguagem de programação influenciam, mas podemos afirmar que a conformidade com o estilo fará com que qualquer API, adquira propriedades desejáveis, como melhora no desempenho, maior escalabilidade, simplicidade, facilidade de manutenção, portabilidade e confiabilidade.

3.2.8 O Mock Fiap-on

Como comentamos, a plataforma Sandbox fornece uma maneira simples de criar um mock em REST para testes. Como vimos, cada projeto Sandbox possui um URL exclusivo, para enviarmos solicitações para o serviço que simulamos. Em nossa API de exemplo, criamos http://fiap-on.getsandbox.com.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Caso você queira criar seu próprio mock, acesse a plataforma, crie uma conta acesse a API Reference em https://getsandbox.com/docs/http-request-handling ou utilize como base o código JavaScript do nosso exemplo.

A leitura do código comentado também será útil para um melhor entendimento sobre os vários assuntos que abordamos nos textos anteriores. Por favor, considere este código apenas para testes simples, não se esqueça, ele é apenas um mock.

```
/**
 * API REST Sandbox desenvolvida em JavaScript para exemplo do EAD FIAP
 *
 */

// ############### Dados em Memória ################################

// Array com dados utilizado pela API para persistência em memória
state.users =  {"usuarios":[
        { id:1, nome: "Fernando Gouveia Lima", cpf: "999.999.999-99" },
        { id:2, nome: "Fernando Barbosa Lima", cpf: "111.111.111-11" }
    ]}



// ############### HTTP GET ########################################

// Uma rota básica para testar a API
Sandbox.define('/ola', 'GET', function(req, res) {
    // Retorna Teste de API - FIAP ON
    res.send('Teste de API - FIAP ON');
});

// Simula um GET para retornar todos os usuários no array state
Sandbox.define('/controle/usuarios', 'GET', function(req, res) {

    // Retorna todos os usuários do Array o Status Code será 200 OK
    return res.json(state.users);
});

// Simula um GET em um usuário específico, informado via parâmetro id
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
Sandbox.define('/controle/usuarios/{id}', 'GET', function(req, res) {

    // Captura o id em req.params
    var id = Number(req.params.id);

    // Busca o usuário no array
    var user = _.find(state.users.usuarios, { "id": id});

    // Se o usuário não for encontrado, retorna o Status Code 404 Not Found
    // e uma mensagem de erro tratada no Body
    if (!user) {
      return res.json(404, { error: { message: 'Usuario inexistente!'} })
    }

    // Se encontrado, retorna o usuário no Body e Status Code padrão
    // no caso 200 OK
    return res.json(user);
});

// ############### HTTP PUT ########################################

Sandbox.define('/controle/usuarios/{id}', 'PUT', function(req, res) {
  // Busca o usuário no array pelo id
  var user = _.find(state.users.usuarios, { 'id': Number(req.params.id) })

  // Se o usuário não for encontrado, retorna o Status Code 404 Not Found
  // e uma mensagem de erro tratada no Body
  if (!user) {
      return res.json(404, { error: { message: 'Usuário inexistente!'} })
  }

  // Faz o update com _.merge
  _.merge(user, req.body)

  // Apenas para demonstrar como fazer um log
  console.log(req.body);

  // Excluí o usuário do array
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
  state.users.usuarios = _.reject(state.users.usuarios, { 'id':
Number(req.params.id) })

  // Adiciona o usuário atualizado no array
  state.users.usuarios.push(user);

  // Se encontrado, retorna o usuário atualizado no Body e Status Code 200 OK
  return res.json(200, { message: { status: 'OK'}, usuario: user });
})

// ############### HTTP POST ##########################################

// Simula a criação de um usuário
Sandbox.define('/controle/usuarios', 'POST', function(req, res) {
    // Retorna o array ou inicializa um array vazio
    state.users = state.users || [];

    // Adiciona um usuário no array state
    state.users.usuarios.push(req.body);

    //Adiciona a URI do novo recurso no cabeçalho HTTP Location
    res.set('Location',
               'http://fiap-on.getsandbox.com/controle/usuarios/'+req.body.id);

    //Retorna o Status Code 201 Created e o usuário adicionado no array
    return res.json(201, { message: { status: 'Created'}, usuario: req.body });
});
// ############### HTTP DELETE ########################################

// Simula um DELETE em um usuário específico, informado via parâmetro id
Sandbox.define('/controle/usuarios/{id}', 'DELETE', function(req, res) {

  // Busca o usuário no array pelo id
  var user = _.find(state.users.usuarios, { 'id': Number(req.params.id) })

  // Se o usuário não for encontrado, retorna o Status Code 404 Not Found
  // e uma mensagem de erro tratada no Body
  if (!user) {
      return res.json(404, { error: { message: 'Usuário inexistente!'} })
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
  }


  // Remove o usuário
  state.users.usuarios = _.reject(state.users.usuarios, { 'id':
Number(req.params.id) })


  // Se encontrado, retorna o Status Code 204 No Content
  // informando que processou com êxito, mas não há o que retornar no Body
  return res.json(204, { message: { status: 'OK'}});
})
```

Código-fonte 3.8 – Mock fiap-on SandBox
Fonte: Elaborado pelo autor (2017)

3.3 REST em java

3.3.1 JAX-RS

A linguagem de programação Java possui uma especificação de nome JAX-RS, Java™ API for RESTful Web Services para definir um conjunto de APIs Java exclusivas para o desenvolvimento de Web Services, codificados de acordo com o estilo arquitetônico REST.

Na escrita deste material, a JAX-RS estava na versão 2.1, era mantida pelo JCP na JSR-000370 JavaTM API for RESTful Web Services (JAX-RS) e podia ser obtida em https://jcp.org/aboutJava/communityprocess/final/jsr370/index.html.

Como as demais especificações Java, a JAX-RS possui uma implementação de referência, no caso o Jersey. Você pode e deve explorar mais, acessando a página https://jersey.github.io

O Jersey RESTful Web Services é um framework de código aberto para desenvolvimento de Web Services RESTful em Java que fornece suporte para APIs JAX-RS. Ele é citado como a implementação de referência para JAX-RS (JSR 311 e JSR 339).

Apesar de não mencionar a JSR 370 em sua página, o Jersey é apontado pelo JCP como implementação de referência na documentação da versão final do JSR 370, link citado acima e na página do JCP https://www.jcp.org/en/jsr/detail?id=370.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Com as informações sobre REST, escolhidas a especificação e a implementação de referência, estamos aptos a desenvolver um serviço RESTful em JAVA que realize operações CRUD.

3.3.2 Serviço Fiap-on em JAX-RS, Jersey e JPA

3.3.2.1 Criando o Projeto

No Eclipse crie um novo Dynamic Web Project:

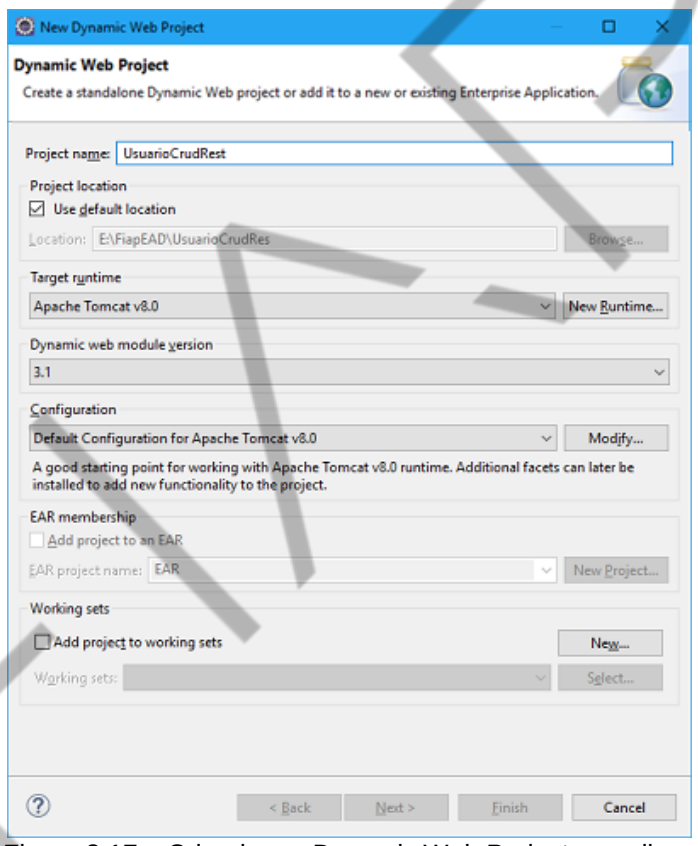

Figura 3.17 – Criando um Dynamic Web Project no eclipse
Fonte: Elaborado pelo autor (2017)

Clique em Next na tela Dynamic Web Project e a tela abaixo aparecerá. Clique em Next novamente.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

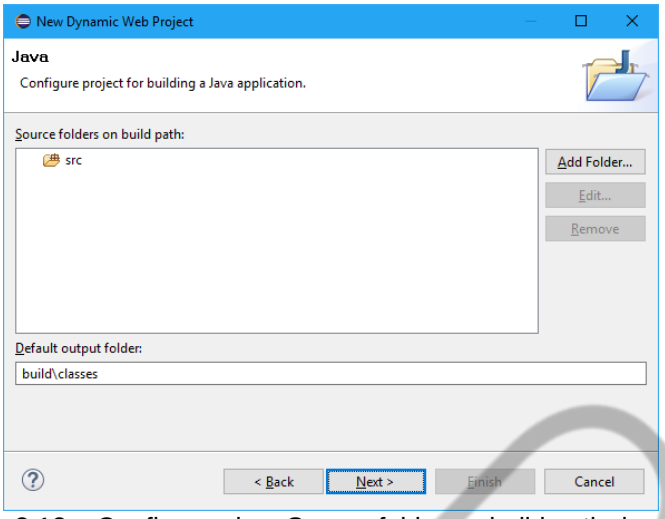

Figura 3.18 – Configurando o Source folder no build path do projeto
Fonte: Elaborado pelo autor (2017)

Mantenha a configuração em “Configure web module settings” para gerar o arquivo web.xml e novamente clique em Next.

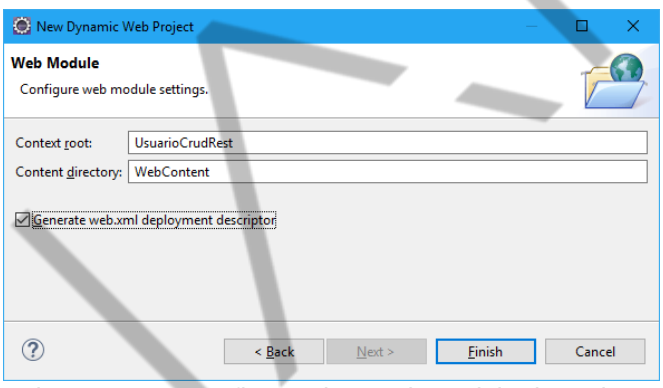

Figura 3.19 – Configurando o web module do projeto
Fonte: Elaborado pelo autor (2017)

O projeto usuarioCrudRest será criado no Eclipse com a seguinte estrutura.

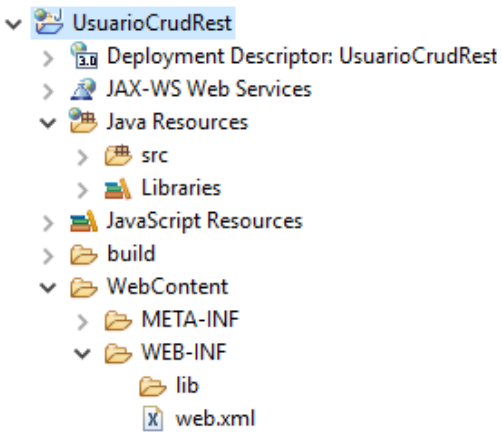

Figura 3.20 – Projeto UsuarioCrudRest no Projeto Explorer Eclipse
Fonte: Elaborado pelo autor (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Adicione o jar do Oracle na pasta lib do seu projeto e adicione-o ao Build Path do projeto.

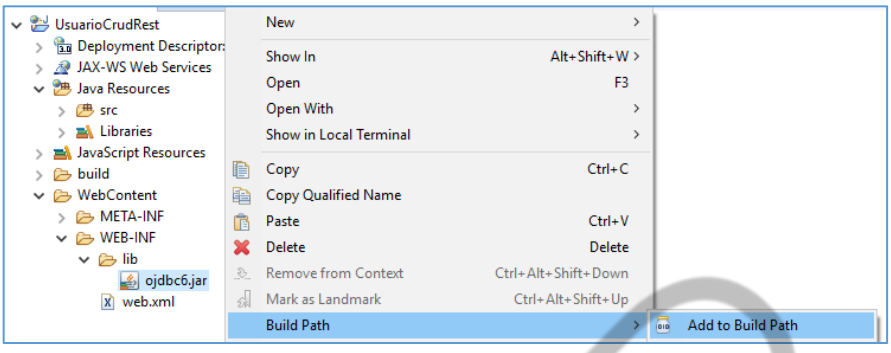

Figura 3.21 – Adicionando o jar Oracle no projeto
Fonte: Elaborado pelo autor (2017)

3.3.2.2 Convertendo o Projeto para Maven Project

Clique com o botão direito sobre o seu projeto e converta-o para Maven Project.

Figura 3.22 – Convertendo o projeto ClienteCrudSoap para Maven Project
Fonte: Elaborado pelo autor (2017)

Configure o POM do seu projeto e clique em Finish.

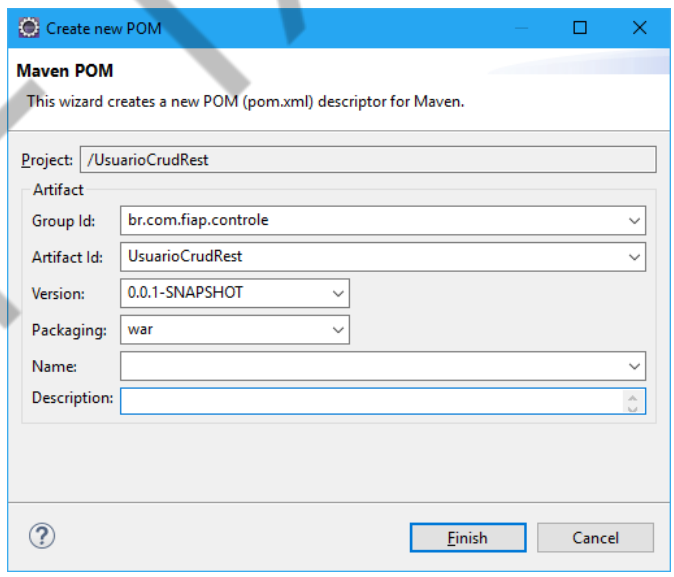

Figura 3.23 – Criando o arquivo pom do projeto
Fonte: Elaborado pelo autor (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

No Project Explorer do Eclipse, clique no arquivo pom.xml gerado pelo wizard e adicione as propriedades e dependências necessárias para usarmos Jersey, JPA e configurarmos um pool de conexão.

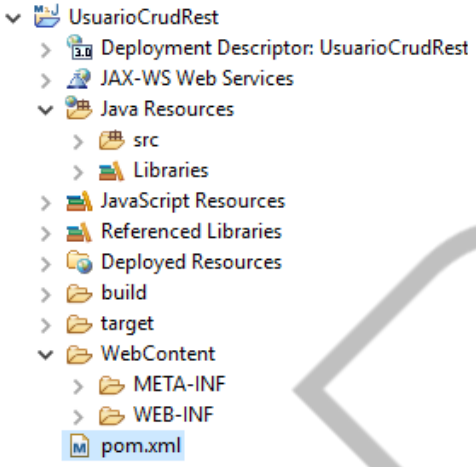

Figura 3.24– O arquivo pom.xml do projeto no Project Explorer do Eclipse
Fonte: Elaborado pelo autor (2017)

3.3.3 Adicionando as dependências Maven

Arquivo pom.xml do projeto.

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
     http://maven.apache.org/xsd/maven-4.0.0.xsd">

     <modelVersion>4.0.0</modelVersion>
      <groupId>br.com.fiap.controle</groupId>
      <artifactId>UsuarioCrudRest</artifactId>
      <version>0.0.1-SNAPSHOT</version>
      <packaging>war</packaging>

      <properties>
            <java.version>1.8</java.version>
            <hibernate.version>
                  5.2.4.Final
            </hibernate.version>
            <jersey.version>
                  2.25.1
            </jersey.version>
            <project.build.sourceEncoding>
                  UTF-8
            </project.build.sourceEncoding>
      </properties>

      <dependencies>

            <!-- Jersey -->
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
        <dependency>
              <groupId>org.glassfish.jersey.core</groupId>
              <artifactId>jersey-server</artifactId>
              <version>${jersey.version}</version>
        </dependency>

        <dependency>
              <groupId>
                org.glassfish.jersey.containers
            </groupId>
              <artifactId>
                jersey-container-servlet
            </artifactId>
              <version>${jersey.version}</version>
        </dependency>

        <dependency>
              <groupId>
               org.glassfish.jersey.media
            </groupId>
              <artifactId>
               jersey-media-json-jackson
              </artifactId>
              <version>${jersey.version}</version>
        </dependency>


        <!-- JPA -->
        <dependency>
              <groupId>org.hibernate</groupId>
              <artifactId>hibernate-core</artifactId>
              <version>${hibernate.version}</version>
        </dependency>

        <dependency>
              <groupId>org.hibernate</groupId>
              <artifactId>
               hibernate-entitymanager
              </artifactId>
              <version>${hibernate.version}</version>
        </dependency>

        <!-- Para configuração do connection pooling -->
        <dependency>
              <groupId>org.hibernate</groupId>
              <artifactId>hibernate-c3p0</artifactId>
              <version>${hibernate.version}</version>
        </dependency>
    </dependencies>


    <build>
        <sourceDirectory>src</sourceDirectory>
        <plugins>
              <plugin>
                    <artifactId>
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
                        maven-compiler-plugin
                      </artifactId>
                       <version>3.6.1</version>
                       <configuration>
                            <source>1.8</source>
                            <target>1.8</target>
                       </configuration>
                 </plugin>
                 <plugin>
                       <artifactId>maven-war-plugin</artifactId>
                       <version>3.0.0</version>
                       <configuration>
                            <warSourceDirectory>
                            WebContent
                          </warSourceDirectory>
                       </configuration>
                 </plugin>
            </plugins>
      </build>
      </project>
```

Código-fonte 3.9 – pom.xml
Fonte: Elaborado pelo autor (2017)

3.3.4 Configurando o JPA

Agora crie a pasta UsuarioCrudRest / Java Resources / src / META-INF e, nela, crie o arquivo persistence.xml com o código na sequência da imagem.

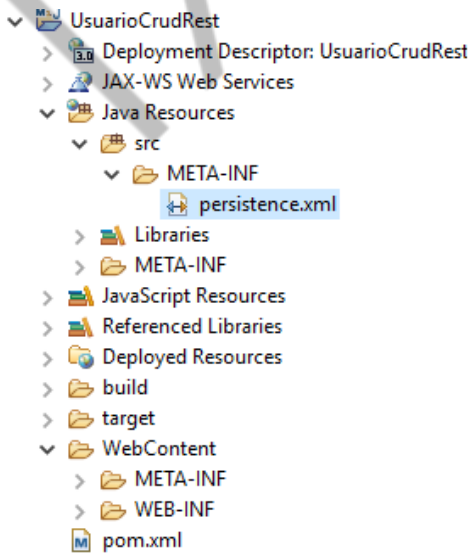

Figura 3.25 – O arquivo pom.xml do projeto no Project Explorer do Eclipse
Fonte: Elaborado pelo autor (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence version="2.1"
      xmlns="http://xmlns.jcp.org/xml/ns/persistence"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/persistence

      http://xmlns.jcp.org/xml/ns/persistence/persistence_2_1.xsd">
      <persistence-unit name="ORACLE_FIAP">

      <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>

            <properties>
                  <property name="hibernate.show_sql" value="true" />
                  <property name="hibernate.format_sql" value="true" />
                  <property name="hibernate.hbm2ddl.auto" value="update"
/>
                  <property name="hibernate.dialect"

value="org.hibernate.dialect.Oracle10gDialect" />
                  <property name="javax.persistence.jdbc.driver"
                              value="oracle.jdbc.OracleDriver" />
                  <property name="javax.persistence.jdbc.user"
value="XXXX" />
                  <property name="javax.persistence.jdbc.password"
value="YYYY" />
                  <property name="javax.persistence.jdbc.url"

value="jdbc:oracle:thin:@oracle.fiap.com.br:1521:orcl" />

                  <!-- Connection Pool -->
                  <!-- Número mínimo de conexões  -->
                  <property name="hibernate.c3p0.min_size" value="5" />
                  <!-- Número máximo de conexões  -->
                  <property name="hibernate.c3p0.max_size" value="20" />
                  <!-- Tempo de timeout, remoção da conexão do pool -->
                  <property name="hibernate.c3p0.timeout" value="500" />
                  <!-- Comandos em cache para aumento de performance -->
                  <property name="hibernate.c3p0.max_statements"
value="50" />
                  <!-- Tempo para uma conexão ser validada
automaticamente -->
                  <property name="hibernate.c3p0.idle_test_period"
                              value="2000" />
            </properties>
      </persistence-unit>
</persistence>
```

Código-fonte 3.10 – persistence.xml
Fonte: Elaborado pelo autor (2017)

Troque o usuário XXXX e a senha YYYY, com o seu RM e respectiva senha do Oracle FIAP.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3.3.5 Criando o back-end do Serviço

Crie a classe EntityManagerFactorySingleton e implemente o código de acordo com o conhecimento adquirido na disciplina Enterprise Application Development.

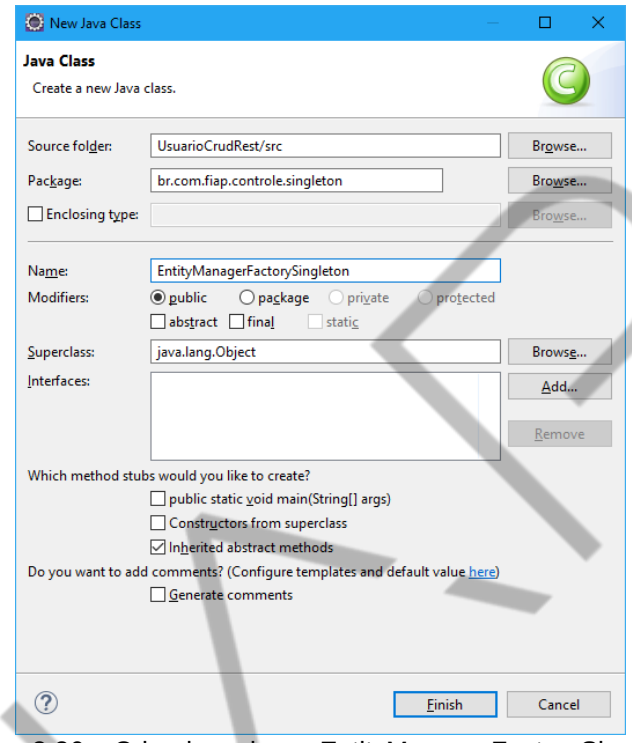

Figura 3.26 – Criando a classe EntityManagerFactorySingleton
Fonte: Elaborado pelo autor (2017)

```
package com.fiap.bank.singleton;

import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;

public class EntityManagerFactorySingleton {

	private static EntityManagerFactory fabrica;
	private EntityManagerFactorySingleton(){}

	public static EntityManagerFactory getInstance(){
		if (fabrica == null){
			fabrica = Persistence
				.createEntityManagerFactory("ORACLE_FIAP");
		}
		return fabrica;
	}
}
```

Código-fonte 3.11 – EntityManagerFactorySingleton.java
Fonte: Elaborado pelo autor (2017)

Acesse o Oracle FIAP e cria uma sequence de nome SEQ_USUARIO.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

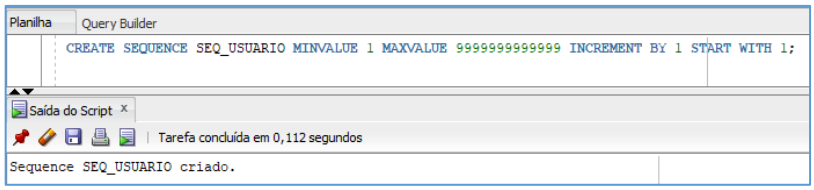

Figura 3.27 – Criando a sequence SEQ_USUARIO no Oracle FIAP
Fonte: Elaborado pelo autor (2017)

Crie a classe Usuario no pacote br.com.fiap.controle.entity.

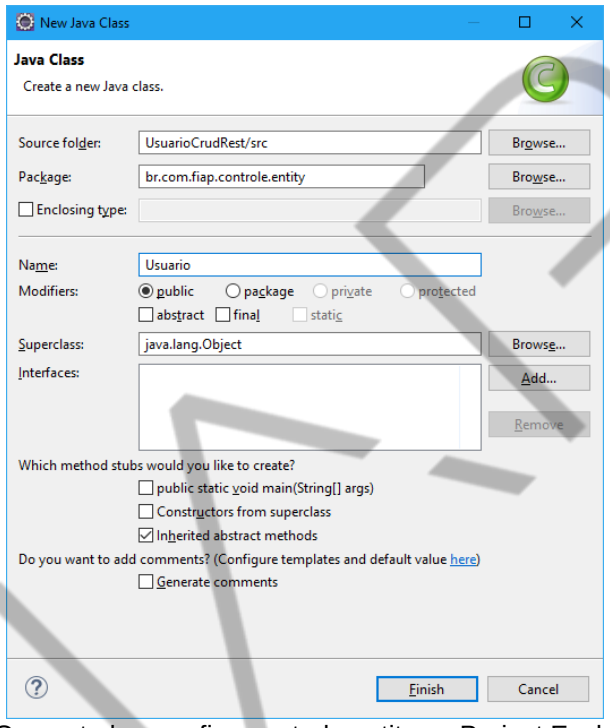

Figura 3.28 – O pacote br.com.fiap.controle.entity no Project Explorer do Eclipse
Fonte: Elaborado pelo autor (2017)

Crie a classe Usuario.java adicionando annotations e imports necessários para a persistência JPA e annotations JAXB, pois esse recurso será exposto como XML além de JSON, pelo nosso Web Service.

A coluna ds_email será criada com a constraint unique, para testarmos um conflito de criação em nosso serviço RESTful.

```
package br.com.fiap.controle.entity;

import java.io.Serializable;
import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.SequenceGenerator;
import javax.persistence.Table;
import javax.xml.bind.annotation.XmlElement;
import javax.xml.bind.annotation.XmlRootElement;
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
@Entity
@Table(name="TBL_CTR_USUARIO")
@SequenceGenerator(name="sqUsuario", sequenceName="SEQ_USUARIO",
allocationSize=1)
@XmlRootElement(name = "Usuario")
public class Usuario implements Serializable{

      private static final long serialVersionUID = 1L;

      @Id
      @GeneratedValue(strategy=GenerationType.SEQUENCE,
                                          generator="sqUsuario")
      @Column(name = "cd_usuario")
      private int idUsuario;
      @Column(name = "nm_usuario", nullable = false, length = 100)
      private String nome;
      @Column(name = "ds_email",
                      nullable = false, unique=true, length = 100)
      private String email;

      public Usuario(int idUsuario, String nome, String email) {
            super();
            this.idUsuario = idUsuario;
            this.nome = nome;
            this.email = email;
      }

      public Usuario(){}

      @XmlElement(name = "idUsuario", required = true)
      public int getIdUsuario() {
            return idUsuario;
      }

      public void setIdUsuario(int idUsuario) {
            this.idUsuario = idUsuario;
      }

      @XmlElement(name = "nome", required = true)
      public String getNome() {
            return nome;
      }
      public void setNome(String nome) {
            this.nome = nome;
      }


      @XmlElement(name = "email", required = true)
      public String getEmail() {
            return email;
      }
      public void setEmail(String email) {
            this.email = email;
      }
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
	@Override
	public String toString() {
		return "Usuario [idUsuario="
						+ idUsuario
						+ ", nome="
						+ nome
						+ ", email="
						+ email
						+ "]";
	}
}
```

Código-fonte 3.12 – Usuario.java
Fonte: Elaborado pelo autor (2017)

Crie uma classe Teste no pacote br.com.fiap.controle.test e execute-a, para testar a persistência da entidade Usuario no Oracle FIAP.

```
package br.com.fiap.controle.test;

import javax.persistence.EntityManager;
import br.com.fiap.controle.entity.Usuario;
import br.com.fiap.controle.singleton.EntityManagerFactorySingleton;

public class Teste {

	public static void main(String[] args) {

		EntityManager em = null;

		Usuario Usuario = new Usuario();
		Usuario.setNome("Fernando Gouveia Lima");
		Usuario.setEmail("qualquer@gmail.com");

		try {
			em = EntityManagerFactorySingleton
								.getInstance()
								.createEntityManager();
			em.getTransaction().begin();
			em.persist(Usuario);
			em.getTransaction().commit();
		} catch (Exception e) {
			if (em.getTransaction().isActive()) {
				em.getTransaction().rollback();
			}
			e.printStackTrace();
			System.out.println("Cadastro não realizado");
		} finally {
			em.close();
		}
	}
}
```

Código-fonte 3.13 – Teste.java
Fonte: Elaborado pelo autor (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Entidade criada, com as devidas constraints e usuário inserido com êxito.

```
Console  Markers  Properties  Servers  Data Source Explorer  Snippets
<terminated> Teste [Java Application] C:\Program Files\Java\jre1.8.0_131\bin\javaw.exe (1 de nov de 2017 22:47:07)
    create table TBL_CTR_USUARIO (
        cd_usuario number(10,0) not null,
        ds_email varchar2(100 char) not null,
        nm_usuario varchar2(100 char) not null,
        primary key (cd_usuario)
    )
Hibernate:

    alter table TBL_CTR_USUARIO
        drop constraint UK_ii8ll5aqp72c4dxuo27q1p74k
Hibernate:

    alter table TBL_CTR_USUARIO
        add constraint UK_ii8ll5aqp72c4dxuo27q1p74k unique (ds_email)
Hibernate:
    select
        SEQ_USUARIO.nextval
    from
        dual
Hibernate:
    insert
    into
        TBL_CTR_USUARIO
        (ds_email, nm_usuario, cd_usuario)
    values
        (?, ?, ?)
```

Figura 3.29 – Log de execução de teste do Hibernate no Console Eclipse
Fonte: Elaborado pelo autor (2017)

FIAP TBL_CTR_USUARIO

Colunas | Dados | Model | Constraints | Concessões | Estatísticas | Triggers | Flashback | Dependências | Detalhes | Partições | Índices | SQL

Classificar.. | Filtro:

	CD_USUARIO	DS_EMAIL	NM_USUARIO
1	1	qualquer@gmail.com	Fernando Gouveia Lima

Figura 3.30 – Confirmação de gravação no Oracle, via acesso pelo Oracle Developer
Fonte: Elaborado pelo autor (2017)

Teste bem-sucedido, vamos criar a camada DAO do nosso serviço. Para tanto, vamos utilizar as classes DataBaseException, EntityNotFoundException, GenericDAO e GenericDAOImpl que você desenvolveu na disciplina Enterprise Application Development. Crie os pacotes e as classes conforme a imagem abaixo.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

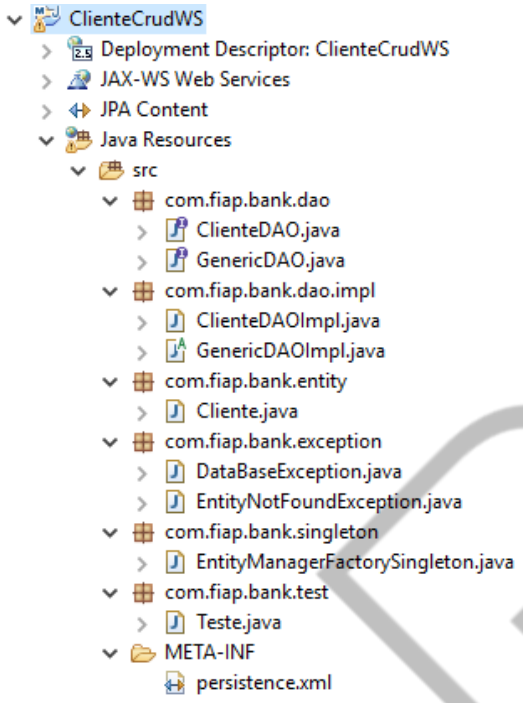

Figura 3.31 – Criação de pacotes de persistência no Project Explorer do Eclipse
Fonte: Elaborado pelo autor (2017)

Desenvolva a interface UsuarioDAO e a classe UsuarioDAOImpl conforme os códigos seguintes.

```
package br.com.fiap.controle.dao;
import br.com.fiap.controle.entity.Usuario;

public interface UsuarioDAO
      extends GenericDAO<Usuario, Integer>{

}
```

Código-fonte 3.14 – UsuarioDAO.java
Fonte: Elaborado pelo autor (2017)

```
package br.com.fiap.controle.dao.impl;

import br.com.fiap.controle.dao.UsuarioDAO;
import br.com.fiap.controle.entity.Usuario;

public class UsuarioDAOImpl
      extends GenericDAOImpl<Usuario, Integer>
      implements UsuarioDAO{
}
```

Código-fonte 3.15 – UsuarioDaoImpl.java
Fonte: Elaborado pelo autor (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3.3.6 Configurando o JAX-RS

Agora, vamos configurar a nossa aplicação para utilizar o JAX-RS, mapeando o servlet do Jersey e apontando uma classe de configuração, a ServicesConfig.java. Altere o arquivo web.xml em WebContent \ WEB-INF da seguinte forma.

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xmlns="http://java.sun.com/xml/ns/javaee"
      xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
       http://java.sun.com/xml/ns/javaee/web-app_3_0.xsd"
      id="WebApp_ID" version="3.0">
      <display-name>UsuarioCrudRest</display-name>
      <servlet>
            <servlet-name>Jersey Servlet</servlet-name>
            <servlet-class>
               org.glassfish.jersey.servlet.ServletContainer
            </servlet-class>
            <init-param>
                  <param-name>
                    javax.ws.rs.Application
                  </param-name>
                  <param-value>
                     br.com.fiap.controle.services.ServicesConfig
                 </param-value>
            </init-param>
      </servlet>
      <servlet-mapping>
            <servlet-name>Jersey Servlet</servlet-name>
            <url-pattern>/*</url-pattern>
      </servlet-mapping>
</web-app>
```

Código-fonte 3.16 – web.xml
Fonte: Elaborado pelo autor (2017)

O parâmetro javax.ws.rs.Application aponta a classe ServicesConfig.java que configurará a utilização do JAX-RS, portanto, crie-a no pacote br.com.fiap.controle.services, com o seguinte código.

```
package br.com.fiap.controle.services;

import org.glassfish.jersey.server.ResourceConfig;

public class ServicesConfig extends ResourceConfig  {
      public ServicesConfig() {
            packages("br.com.fiap.controle.services");
     }
}
```

Código-fonte 3.17 – ServicesConfig.java
Fonte: Elaborado pelo autor (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

O web.xml mapeou o Servlet do Jersey para assumir o processamento em qualquer URL solicitada a partir de localhost:8080/UsuarioCrudRest/, por meio da configuração <url-pattern>/*</url-pattern>. Como comentamos, o arquivo web.xml também determinou a classe ServicesConfig que estende a classe ResourceConfig do pacote org.glassfish.jersey.server, para configurar os serviços da aplicação.

Em comparação com classes de configuração que estendem Application do javax.ws.rs.core, ResourceConfig fornece recursos avançados para simplificar o registro de componentes do JAX-RS, como a varredura no package br.com.fiap.controle.services para registro automático dos serviços que estiverem dentro dele.

3.3.7 Codificando e testando o Serviço de forma básica

Neste ponto, a sua classe ServicesConfig.java não compila, pois ainda não criamos a classe que será o nosso Web Service JAX-RS. Então, crie-a no pacote br.com.fiap.controle.services, com uma abordagem inicial muito conhecida.

```
package br.com.fiap.controle.services;

import javax.ws.rs.GET;
import javax.ws.rs.Path;
import javax.ws.rs.Produces;

@Path("/controle")
public class UsuarioService {

	@GET
	@Path("/ola")
	@Produces("text/plain")
	public String testaServico() {
		return "Olá, o serviço funciona";
	}
}
```

Código-fonte 3.18 – UsuarioService.java
Fonte: Elaborado pelo autor (2017)

Ao executarmos a nossa aplicação no Tomcat, o serviço ficará disponível para solicitações GET na URL http://localhost:8088/UsuarioCrudRest/controle/ola. Perceba que o serviço funciona, mas não como gostaríamos.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

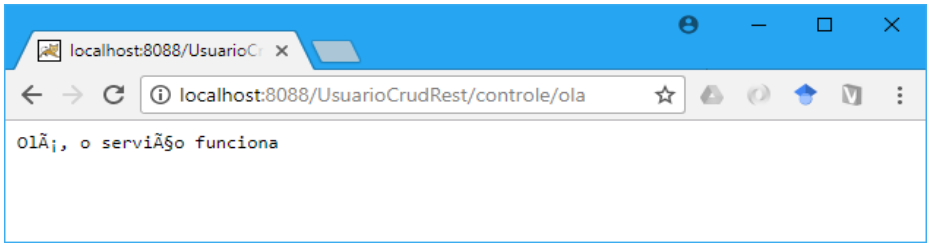

Figura 3.32 – Teste da API - /controle/ola
Fonte: Elaborado pelo autor (2017)

Altere @Produces("text/plain") para @Produces("text/plain;charset=utf-8") e execute o serviço novamente.

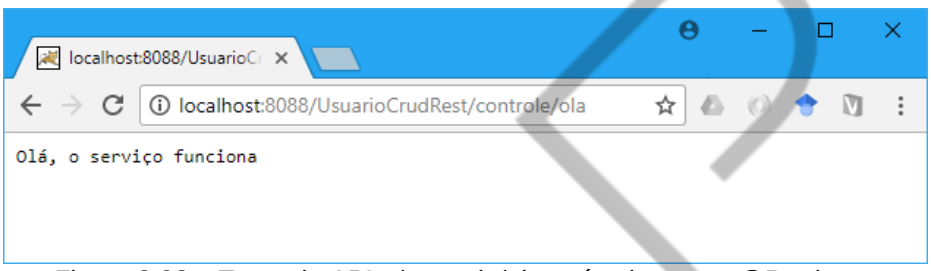

Figura 3.33 – Teste da API - /controle/ola após ajuste em @Produces
Fonte: Elaborado pelo autor (2017)

Neste ponto, creio que você já percebeu como configurações adicionais aos Media Types podem ser importantes. Sem a codificação binária UTF-8, tivemos sérios problemas com caracteres acentuados.

Apesar de muito simples, nosso serviço básico foi codificado utilizando conceitos importantes. Todas as definições do nosso serviço JAX-RS foram feitas por meio de anotações Java. A anotação @Path determinou a URL protocolo://servidor:porta/nome da aplicação/**controle/ola**, já a anotação @GET, o verbo HTTP pelo qual o método testaServico() nos atendeu e @Produces determinou o Media Type que o método do serviço retornará.

Perceba que **/controle/ola,** em conjunto com servidor + porta + nome da aplicação, gera uma URL única para o método testaServico(). Como boa prática de segurança, sempre utilize um nome diferente do método Java, para chamá-lo pela URI. Em nosso exemplo, utilizamos /ola em vez de /testaServico.

3.3.8 Consultando todos os usuários – HTTP GET

Chegou o momento de nosso serviço acessar o banco de dados. Para tanto, ainda temos que fazer uma pequena alteração no nosso back-end. Dentro do pacote

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

br.com.fiap.controle.entity.wrapper, crie uma classe para conter (encapsular) todos os usuários retornados da base, implemente-a com o código abaixo.

Apesar de muito simples, essa classe wrapper será a responsável por determinar o elemento raiz <usuarios></usuarios> no XML e o array usuarios:[ ] no JSON.

```
package br.com.fiap.controle.entity.wrapper;

import java.util.ArrayList;
import java.util.List;
import javax.xml.bind.annotation.XmlElement;
import javax.xml.bind.annotation.XmlRootElement;

import br.com.fiap.controle.entity.Usuario;

@XmlRootElement
public class Usuarios {

      private List<Usuario> usuarios = new ArrayList<>();

      public Usuarios() {   }


      public Usuarios(List<Usuario> usuarios) {
            this.usuarios = usuarios;
      }


      @XmlElement(name = "usuario")
      public List<Usuario> getUsuarios() {
            return usuarios;
      }

      public void setUsuarios(List<Usuario> usuarios) {
            this.usuarios = usuarios;
      }

}
```

Código-fonte 3.19 – Usuarios.java
Fonte: Elaborado pelo autor (2017)

Agora, em nosso Serviço, vamos codificar o método buscarTodosUsuarios() para retornar os usuários da nossa base no Oracle FIAP, representados nos Medias Types XML ou JSON. Altere a classe UsuarioService, conforme código abaixo.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```java
package br.com.fiap.controle.services;

import javax.ws.rs.Consumes;
import javax.ws.rs.GET;
import javax.ws.rs.Path;
import javax.ws.rs.Produces;
import javax.ws.rs.WebApplicationException;
import javax.ws.rs.core.MediaType;

import br.com.fiap.controle.dao.UsuarioDAO;
import br.com.fiap.controle.dao.impl.UsuarioDAOImpl;
import br.com.fiap.controle.entity.wrapper.Usuarios;
import br.com.fiap.controle.exception.EntityNotFoundException;

@Path("/controle")
@Consumes({ MediaType.APPLICATION_XML, MediaType.APPLICATION_JSON })
@Produces({ MediaType.APPLICATION_XML, MediaType.APPLICATION_JSON })

public class UsuarioService {

	private UsuarioDAO usuarioDAO = new UsuarioDAOImpl();

	@GET
	@Path("/ola")
	@Produces("text/plain;charset=utf-8")
	public String testaServico() {
		return "Olá, o serviço funciona";
	}

	@GET
	@Path("/usuarios")
	public Usuarios buscarTodosUsuarios() {


		try {
			Usuarios usuarios = new
						Usuarios(usuarioDAO.buscarTodos());
			return usuarios;
		} catch (EntityNotFoundException e) {
			throw new WebApplicationException(404);
		} catch (Exception e) {
			throw new WebApplicationException(500);
		}
	}

}
```

Código-fonte 3.20 – UsuarioService.java
Fonte: Elaborado pelo autor (2017)

A anotação @Produces determinou os Media Types que o serviço retornará e @Consumes os Media Types que receberá, quando um método não possuir @Produces ou @Consumes específicos.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

O método do DAO buscarTodos() retornará um List<Usuario> que será passado ao construtor do wrapper Usuarios, para que o Wrapper seja retornado pelo método buscarTodosUsuarios() do serviço.

Caso não exista nenhum usuário na base, a exceção EntityNotFoundException será lançada pelo método buscarTodos() do DAO e capturada pelo try catch, para lançamento do Status Code 404 - Not Found. Caso um outro tipo de exceção ocorra, o método lançará uma WebApplicationException mais genérica, o Status Code 500 - Internal Server Error.

Agora vamos testar uma característica muito importante de APIs RESTful, o uso de Media Types no cabeçalho Accept para obtermos representações diferentes de um mesmo recurso.

No Postman, faça uma requisição GET, parametrizando em Headers – Key Accept e Value application/json para a seguinte URL: http://localhost:8088/UsuarioCrudRest/controle/usuarios

Como demonstrado na imagem, você obterá um JSON. Perceba que o body possui um array de nome usuarios:[ ], esse array foi criado quando a nossa classe wrapper Usuarios foi representada em JSON.

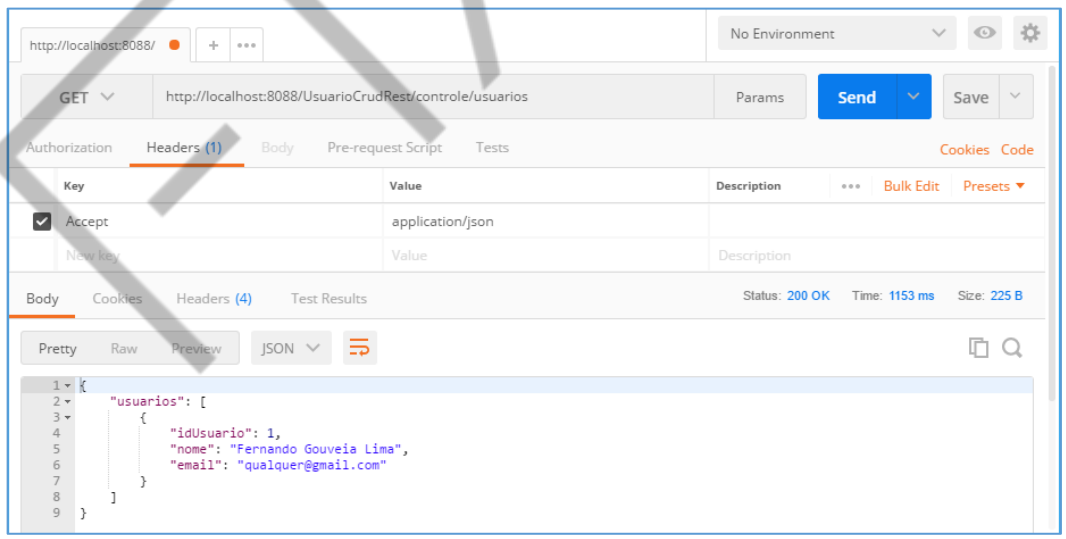

Figura 3.34 – Teste da API - /controle/usuarios – Accept application/json
Fonte: Elaborado pelo autor (2017)

Se alterarmos o Value do cabeçalho Accept para application/xml, nossa API retornará um XML. Note que o elemento raiz usuarios, tal qual no array JSON, será criado na representação da classe wrapper Usuarios, agora em XML.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

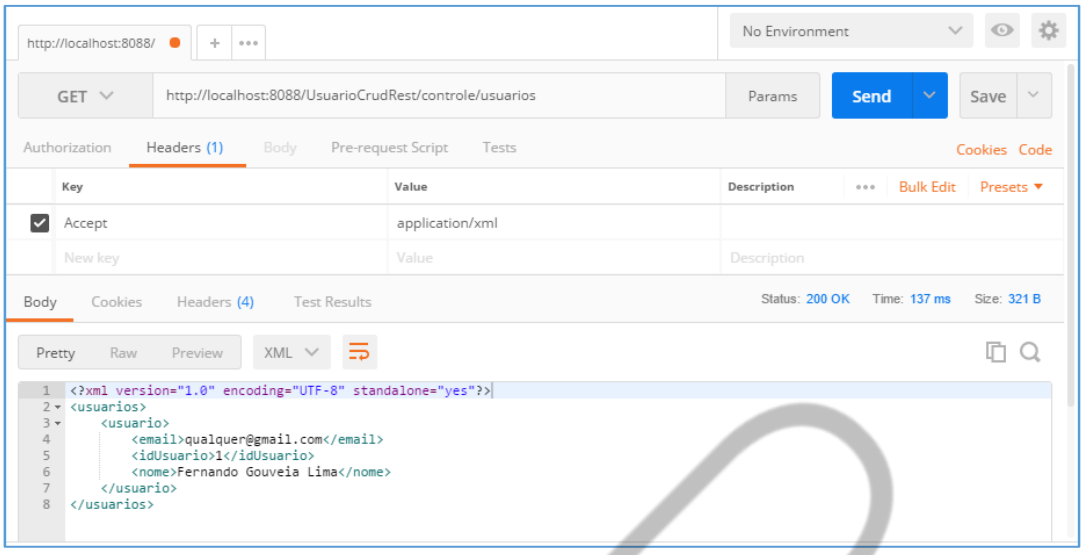

Figura 3.35 – Teste da API - /controle/usuarios – Accept application/xml
Fonte: Elaborado pelo autor (2017)

Exclua o usuário da base Oracle e teste no navegador. Esse será o resultado da exceção WebApplicationException(404) em uma requisição GET no Chrome.

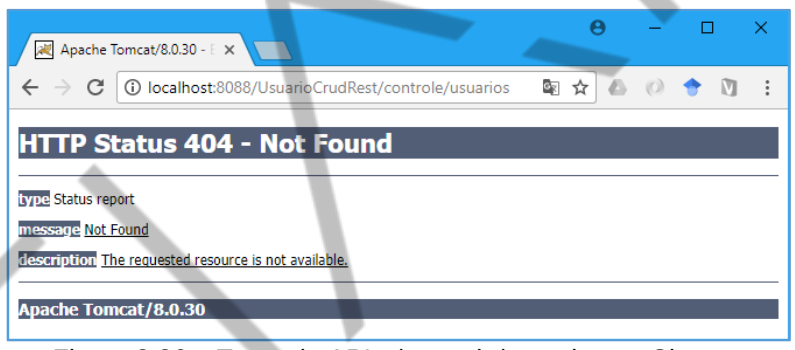

Figura 3.36 – Teste da API - /controle/usuarios no Chrome
Fonte: Elaborado pelo autor (2017)

3.3.9 Consultando e excluindo usuários pelo id – HTTP GET e DELETE

No próximo passo, vamos implementar dois métodos de uma vez. Uma busca por id (GET) e a exclusão de um recurso por id (DELETE), codificaremos os dois juntos, pois ambos possuem o mesmo @Path, obtêm o id do usuário da mesma maneira e realizam os mesmos tratamentos de exceções.

```
@GET
@Path("/usuarios/{id}")
public Usuario buscarUsuarioPorId(@PathParam("id") Integer id) {
      try {
            return usuarioDAO.buscarPorId(id);
      } catch (EntityNotFoundException e) {
            throw new WebApplicationException(Status.NOT_FOUND);
      } catch (Exception e) {
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
            throw new WebApplicationException(

                                    Status.INTERNAL_SERVER_ERROR);
            }
      }

@DELETE
@Path("/usuarios/{id}")
public void removerUsuario(@PathParam("id") Integer id){
      try {
            usuarioDAO.remover(id);
      }catch (EntityNotFoundException e) {
            throw new WebApplicationException(Status.NOT_FOUND);
      }catch (DataBaseException e) {
            throw new WebApplicationException(

                                    Status.INTERNAL_SERVER_ERROR);
      }
}
```

Código-fonte 3.21 – @GET e @DELETE por id em UsuarioService.java
Fonte: Elaborado pelo autor (2017)

No eclipse, use o atalho Ctrl+Shift+O para fazer os imports. Note que a lógica de tratamento de exceções é a mesma do método buscarTodosUsuarios(), mas para facilitar o entendimento, não estamos mais utilizando os números do HTTP Status Codes e sim constantes, obtidos a partir do import javax.ws.rs.core.Response.Status.

Ambos os métodos precisam receber o id do usuário como parâmetro e isso foi possível utilizando a anotação @PathParam, que tem o papel de ligar o {id} da URL /usuarios/{id} ao parâmetro Integer id, nos dois métodos.

Em meio a tantas semelhanças, temos uma diferença sutil, mas muito importante. O método que implementa o DELETE retorna void, ou seja, se nenhuma exceção ocorrer e o recurso for excluído, o Status Code 204 No Content será retornado, informando sobre o êxito no processamento, sem conteúdo no body da mensagem de retorno.

Já o método buscarUsuarioPorId, quando executado com sucesso, retorna um Usuario e o Status Code 200 OK, para informar sobre o êxito no processamento, com uma representação do usuário no body da mensagem de retorno.

3.3.10 Criando um usuário – HTTP POST

Agora, vamos codificar o método para criar um recurso (POST), mas antes vale lembrar as boas práticas REST para esta operação. Como um novo recurso está

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

sendo criado, não seria funcional retornar a URI e uma representação do novo recurso? Creio que sim, pois o id é criado por uma sequence do Oracle, portanto, desconhecido do cliente que solicitou a gravação.

Buscar o novo cliente sem a URI, simplesmente imaginando que ele é o último usuário na base é muito arriscado, imagine que no segundo seguinte ao seu POST, outro usuário foi criado por outra pessoa. Pois bem, veremos como o JAX-RS resolve esses desafios codificando o método abaixo.

```
@POST
@Path("/usuarios")
public Response criarUsuario(Usuario usuario){
      try {
            Usuario u = usuarioDAO.adicionar(usuario);
            URI uri = UriBuilder
                      .fromPath("usuarios/{id}")
                      .build(u.getIdUsuario());

            return Response.created(uri).entity(u).build();

      }catch (DataBaseException e) {
            throw new
                WebApplicationException(Status.CONFLICT);
      }catch (Exception e) {
            throw new
       WebApplicationException(Status.INTERNAL_SERVER_ERROR);
      }
}
```

Código-fonte 3.22 – @Post em UsuarioService.java
Fonte: Elaborado pelo autor (2017)

Não se esqueça, no eclipse, use o atalho Ctrl+Shift+O para fazer os imports.

O método adicionar(usuario), do nosso DAO, cria um usuário no Oracle e o retorna para o objeto u. O UriBuilder cria uma URI, substituindo {id} em “usuarios/{id}” pelo valor do id do novo usuário, por meio do método getIdUsuario() . Ou seja, como exemplo, o segundo usuário criado na base, terá o id = 2, gerado pela sequence Oracle, sendo assim, a URI construída pelo Builder será “usuarios/2” e será nessa URI, que o novo recurso poderá ser obtido por meio de uma solicitação GET.

Na linha seguinte, utilizamos o método created da classe Response, passando como parâmetro a uri criada. No mesmo trecho de código, utilizamos o método entity para retornar o recurso que acabamos de criar. O método created gerará o Status Code 201 Created e adicionará no valor do cabeçalho de resposta Location a URL completa para acessar o novo recurso.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Vamos testar? No Postman, use a URL da imagem e crie dois cabeçalhos, o primeiro para informar que enviaremos um XML via POST e o segundo para solicitar o recurso criado representado em JSON. Claro que essa não é uma prática muito comum, geralmente enviamos e recebemos a mesma representação, mas julgo essa abordagem interessante, para explorarmos a flexibilidade do REST.

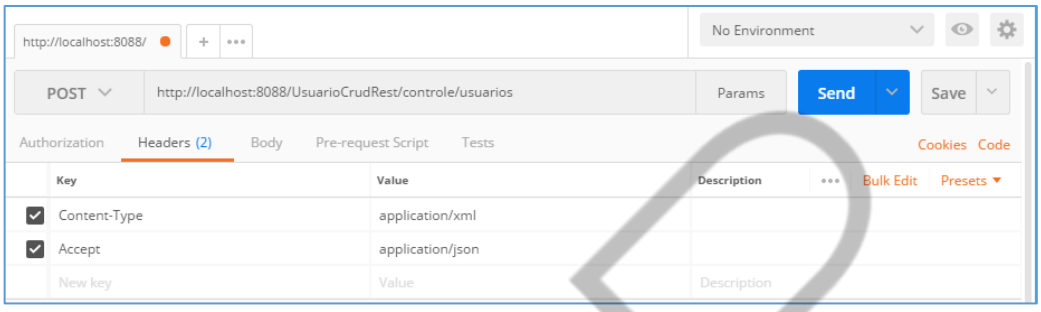

Figura 3.37 – HTTP Request Headers no Postman
Fonte: Elaborado pelo autor (2017)

Em Body, selecione raw e adicione o recurso que queremos criar, utilizando XML. Como o id do usuário será gerado pela sequence, não envie valor nesse elemento.

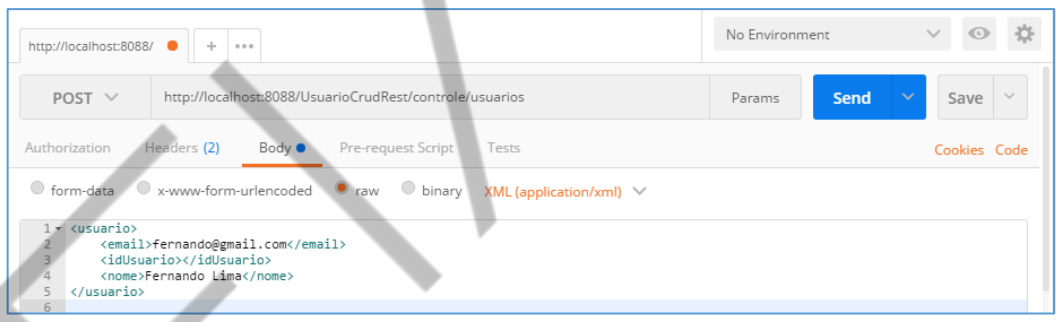

Figura 3.38 – POST Request Body no Postman
Fonte: Elaborado pelo autor (2017)

Como resposta, obteremos o Status Code 201 Created e a representação do recurso criado em JSON, com o id gerado pela sequence Oracle, conforme a imagem abaixo.

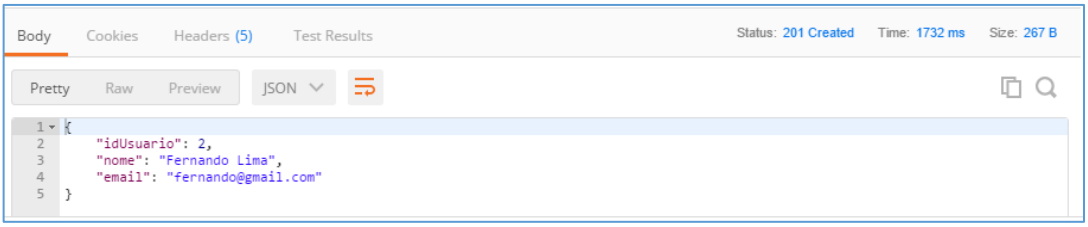

Figura 3.39 – Response Body e Status Code no Postman
Fonte: Elaborado pelo autor (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Em Headers, podemos ver o Content-Type retornado e o Location contendo a URL completa para o novo recurso criado.

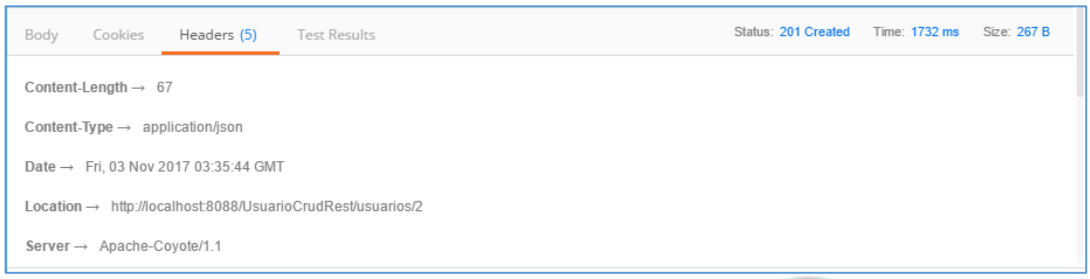

Figura 3.40 – Response Headers - Location no Postman
Fonte: Elaborado pelo autor (2017)

Não podemos nos esquecer que temos mais um teste importante a realizar. Se refizermos o mesmo POST, ou seja, tentarmos gravar um novo usuário com o mesmo e-mail, sabendo que existe uma constraint unique para essa coluna no Oracle, como a aplicação se comportará?

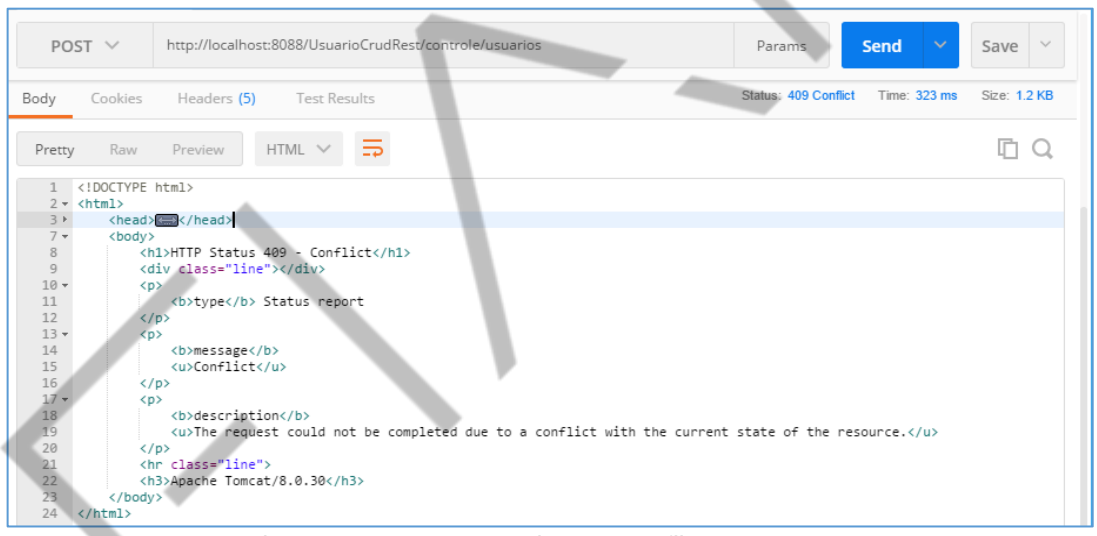

Figura 3.41 – Status Code 409 Conflict no Postman
Fonte: Elaborado pelo autor (2017)

A API se comportou como esperado, não realizou a criação e retornou o Status Code 409 Conflict, informando ao solicitante que a requisição não pode ser completada, devido a um conflito de recursos.

3.3.11 Alterando um usuário – HTTP PUT

Deixamos para o final o método PUT, verbo utilizado para alterarmos ou criarmos um recurso. No código abaixo, vamos codificar o método para apenas alterar um recurso.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
@PUT
@Path("/usuarios/{id}")
public Usuario atualizarUsuario(@PathParam("id") Integer id,
                                                Usuario usuario){
      try {
            buscarUsuarioPorId(id);
            usuario.setIdUsuario(id);
            return usuarioDAO.atualizar(usuario);

      }catch (WebApplicationException WebException) {
            throw WebException;

      }catch(PersistenceException perException) {
            throw new
                  WebApplicationException(Status.CONFLICT);

      }catch (Exception e) {
            throw new
                  WebApplicationException(
                              Status.INTERNAL_SERVER_ERROR);
      }
}
```

Código-fonte 3.23 – @PUT em UsuarioService.java
Fonte: Elaborado pelo autor (2017)

Use o atalho Ctrl+Shift+O para fazer os imports.

Nosso método recebe o id como parâmetro e a representação completa do recurso que será alterado.

Utilizamos o método buscarUsuarioPorId(id) para descobrir se o usuário existe na base, caso não exista, a exceção WebApplicationException será lançada com o `Status.NOT_FOUND` e o nosso método a relançará no catch. Parando a execução e retornando ao solicitador que o usuário não foi localizado, Status Code 404 Not Found.

Se o cliente for localizado, o método setIdUsuario será executado para atribuir o parâmetro id da URL à propriedade id do recurso que recebemos como parâmetro. Essa abordagem garante que o cliente realmente sabe o que quer atualizar.

Para exemplificarmos, imagine que um cliente equivocado faça o seguinte:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

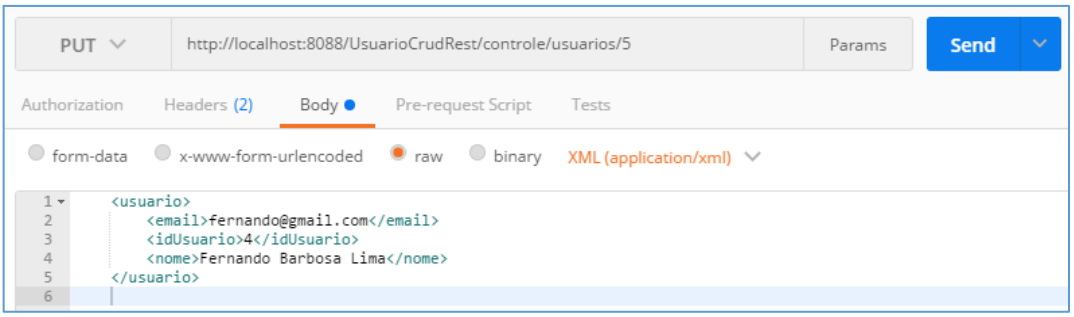

Figura 3.42 – Testando uma solicitação PUT equivocada no Postman
Fonte: Elaborado pelo autor (2017)

Tem algum sentido ele enviar o id 5 na URL e o id 4 no XML? Caso isso aconteça, como o nosso método se comportará?

Ao receber a representação XML, o unmarshall ocorrerá e ele se tornará o objeto usuario (segundo parâmetro do método Java). Na linha usuario.setIdUsuario(id), o método pegará o valor 5 (que chegou na URL) e realizará um set, alterando o valor do atributo id do objeto, de 4 para 5.

Quando o comando usuarioDAO.atualizar(usuario) for executado, o usuário de id 5 será atualizado na base será o e não mais o id 4. Ou seja, o parâmetro id da URL terá prioridade ao valor do id, contido no body.

Ao atualizarmos o recurso, se a constraint unique da coluna ds_email Oracle for disparada, ela será capturada no catch PersistenceException e a API retornará o Status Code 409 Conflict. Para que isso aconteça, uma alteração no back-end será necessária. Garanta que o seu método de atualização lance uma PersistenceException para ser tratada pela API.

```
catch(PersistenceException perException) {
      throw perException;
}
```

Código-fonte 3.24 – catch PersistenceException em GenericDAOImpl.java
Fonte: Elaborado pelo autor (2017)

Como teste, vamos alterar o nome do usuário que criamos quando testamos o método POST.

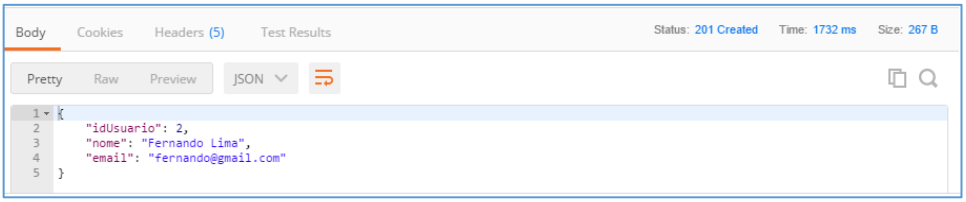

Figura 3.43 – Usuário criado via POST no Postman
Fonte: Elaborado pelo autor (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Desta vez, vamos enviar um JSON e receber um XML, para tanto, configure o método, a URL e os cabeçalhos da requisição no Postman.

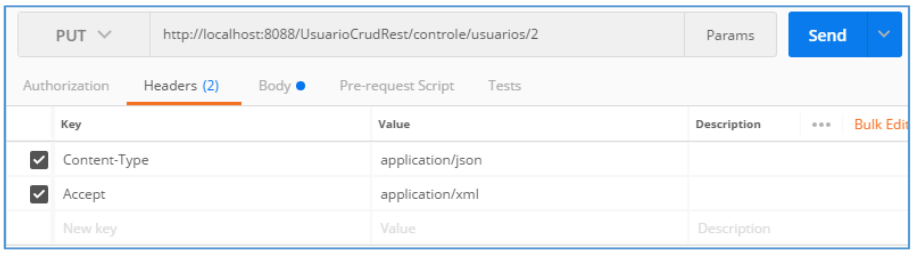

Figura 3.44 – Configurando Headers da requisição PUT no Postman
Fonte: Elaborado pelo autor (2017)

Em Body, selecione raw e adicione a representação em JSON do recurso cujo nome queremos alterar e clique em Send.

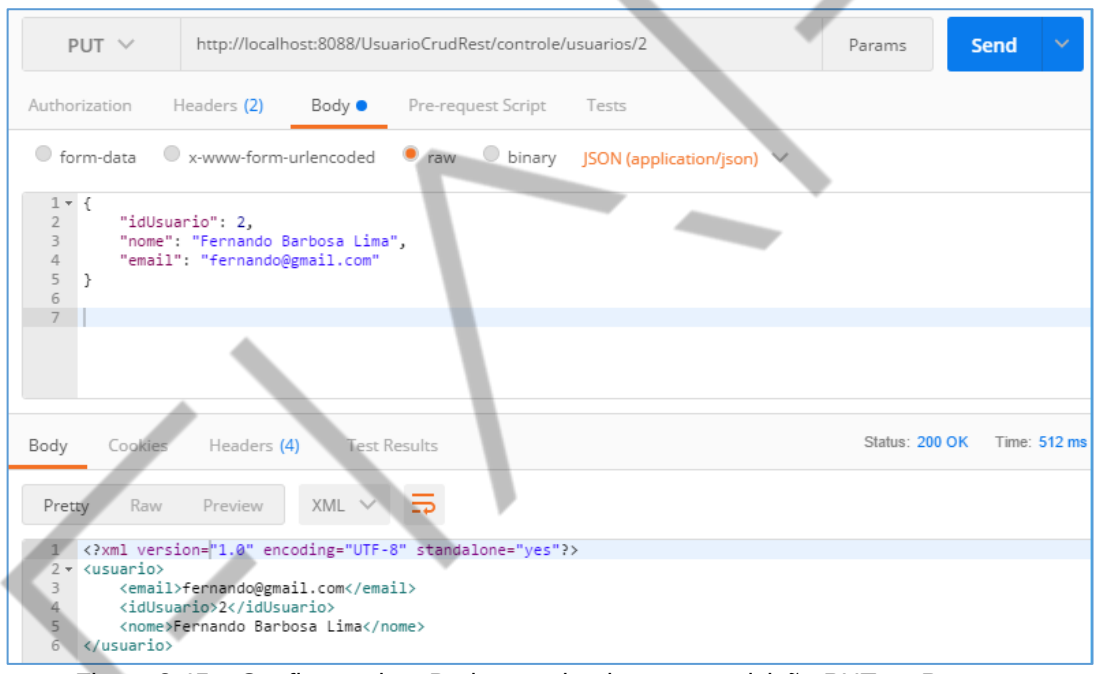

Figura 3.45 – Configurando o Body e enviando uma requisição PUT no Postman
Fonte: Elaborado pelo autor (2017)

Requisição enviada, recebermos o Status Code 200 OK e o recurso representado em XML, de acordo com o cabeçalho Accept da requisição.

Faça os demais testes, tente alterar um usuário que não existe na base ou no usuário de id 2, force um erro de constraint, alterando o email com o mesmo email do usuário de id 1.

Caso você queira que o PUT crie um recurso quando não o encontrar na base, comente ou remova o comando buscarUsuarioPorId(id) e o catch WebApplicationException.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3.3.12 Controlando o acesso à API

Nos passos anteriores, nós criamos uma API RESTful que realiza um CRUD de usuários. Apesar de simples, um serviço com tal característica não ficaria totalmente exposto em uma solução de mercado. Controlar a identificação do cliente e os direitos que ele tem, com certeza, seria um requisito funcional desse tipo de solução.

Faria sentido qualquer cliente que conheça a URL criar um novo usuário? Creio que não. Então, vamos configurar no Tomcat, um mecanismo de controle de acesso e autenticação baseado no HTTP, o HTTP “Basic” authentication.

O RFC 7235 define a estrutura de autenticação HTTP que pode ser usada por um servidor para desafiar uma solicitação de um cliente e, por um cliente, para fornecer informações de autenticação.

O desafio e o fluxo de respostas são muito simples: o cliente que desconhece a obrigatoriedade, faz uma solicitação sem dados de autenticação. O servidor responde ao cliente, com o Status Code 401 (não autorizado) e com o cabeçalho de resposta WWW-Authenticate, preenchido com informações sobre como se autorizar. Então, o cliente pode se autenticar, incluindo um campo Authorization no cabeçalho da nova solicitação, contendo as credenciais corretas.

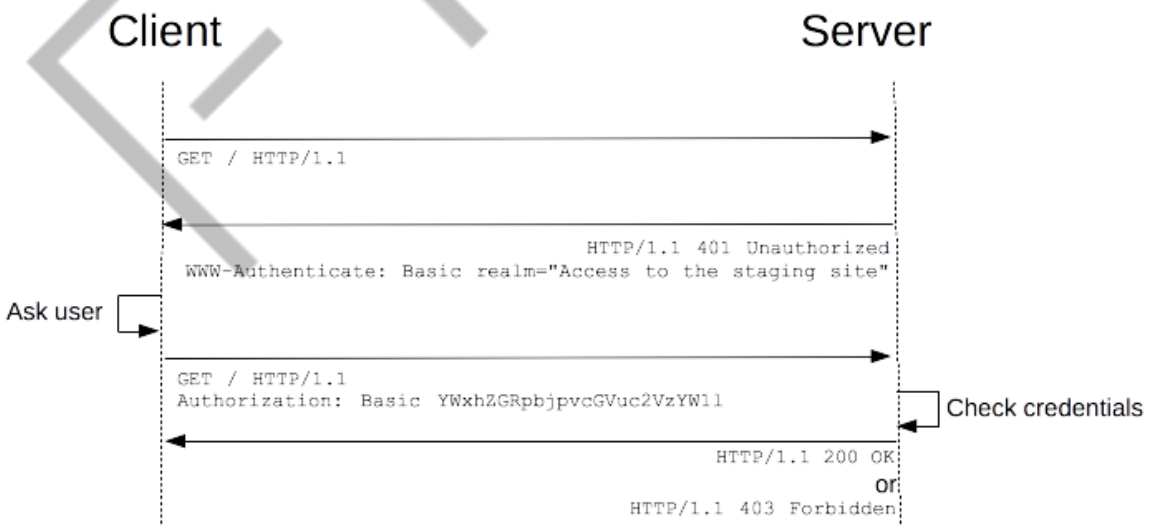

Figura 3.46 – Diagrama de sequência – HTTP Basic Authentication
Fonte: Mozilla (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3.3.12.1 Tomcat Realm

Um Realm é uma fonte de dados que contém nomes de usuários e senhas, utilizadas pelo Tomcat para identificar usuários válidos de um aplicativo da web, além de uma lista de roles (papéis), associados a cada usuário válido.

O Tomcat define uma interface Java (org.apache.catalina.realm) implementada por componentes para estabelecer uma conexão com a fonte de dados escolhida. As fontes de dados mais comuns são arquivos xml, bancos de dados relacionais e servidores de diretórios em LDAP.

3.3.12.2 Autenticação MemoryRealm

Em nosso projeto, entre as opções da interface Realm, utilizaremos o MemoryRealm para acessar informações de autenticação que serão armazenadas em uma coleção de objetos, na memória do servidor. A coleção será inicializada no start do servidor, a partir de do XML de configuração tomcat-users.xml.

No Project Explorer em Servers, abra o arquivo tomcat-users.xml.

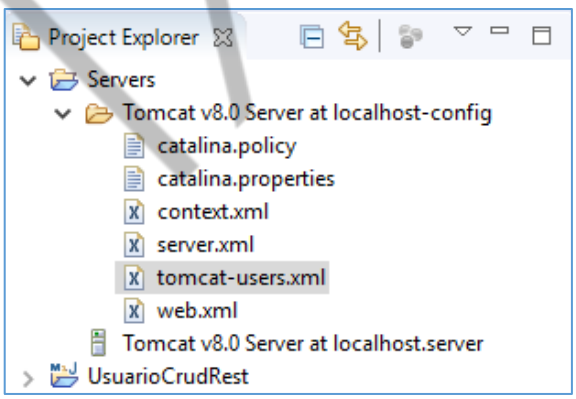

Figura 3.47 – tomcat-users.xml em Servers no Project Explorer do Eclipse
Fonte: Elaborado pelo autor (2017)

Adicione o código entre as tags <tomcat-users ...> ... </tomcat-users>. Esse arquivo possui roles e users comentados como exemplos. Mantenha os trechos comentados, se assim o desejar. Nossa aplicação utilizará os usuários, user, guest e admin e as roles user e guest.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
<tomcat-users version="1.0" xmlns="http://tomcat.apache.org/xml"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://tomcat.apache.org/xml tomcat-users.xs
<!--
  <role rolename="tomcat"/>
  <role rolename="role1"/>
  <user username="tomcat" password="tomcat" roles="tomcat"/>
  <user username="both" password="tomcat" roles="tomcat,role1"/>
  <user username="role1" password="tomcat" roles="role1"/>
-->

  <role rolename="user"/>
  <role rolename="guest"/>
  <user username="user"   password="fiapUser" roles="user"/>
  <user username="guest" password="fiapGuest" roles="guest"/>
  <user username="admin" password="fiapAdmin" roles="user,guest"/>

</tomcat-users>
```

Código-fonte 3.25 – tomcat-users.xml
Fonte: Elaborado pelo autor (2017)

Ainda no Project Explorer do Eclipse, abra o arquivo server.xml, localize e adicione no contexto da aplicação <Context> ... </Context>, a classe que implementará o MemoryRealm - org.apache.catalina.realm.MemoryRealm.

```
<Context docBase="UsuarioCrudRest" path="/UsuarioCrudRest"
reloadable="true" source="org.eclipse.jst.jee.server:UsuarioCrudRest">

  <Realm className="org.apache.catalina.realm.MemoryRealm" />

</Context>
```

Código-fonte 3.26 – server.xml
Fonte: Elaborado pelo autor (2017)

Aplicando <Realm> em <Context>, apenas requisições à aplicação UsuarioCrudRest terão o controle de autenticação pelo container do Tomcat.

O próximo passo é configurar a aplicação para utilizar a implementação MemoryRealm, configurando o arquivo web.xml da nossa aplicação.

Abra o arquivo web.xml em WebContent \ WEB-INF.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

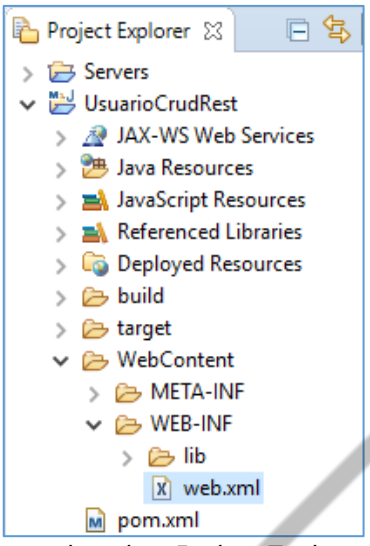

Figura 3.48 – web.xml no Project Explorer do Eclipse
Fonte: Elaborado pelo autor (2017)

Altere-o com o seguinte código:

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xmlns="http://java.sun.com/xml/ns/javaee"
      xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
http://java.sun.com/xml/ns/javaee/web-app_3_0.xsd"
      id="WebApp_ID" version="3.0">
      <display-name>UsuarioCrudRest</display-name>

      <servlet>
            <servlet-name>Jersey Servlet</servlet-name>
            <servlet-class>
                    org.glassfish.jersey.servlet.ServletContainer
          </servlet-class>
            <init-param>
                  <param-name>
                        javax.ws.rs.Application
                  </param-name>
                  <param-value>
                        br.com.fiap.controle.services.ServicesConfig
                  </param-value>
            </init-param>
      </servlet>

      <servlet-mapping>
            <servlet-name>Jersey Servlet</servlet-name>
            <url-pattern>/*</url-pattern>
      </servlet-mapping>

      <security-constraint>
            <web-resource-collection>
                  <web-resource-name>Controle</web-resource-name>
                  <url-pattern>/*</url-pattern>
            </web-resource-collection>
            <auth-constraint>
                  <role-name>*</role-name>
            </auth-constraint>
      </security-constraint>
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
        <security-role>
              <role-name>admin</role-name>
        </security-role>
        <security-role>
              <role-name>guest</role-name>
        </security-role>

        <login-config>
              <auth-method>BASIC</auth-method>
              <realm-name>ControleRM</realm-name>
        </login-config>

</web-app>
```

Código-fonte 3.27 – web.xml
Fonte: Elaborado pelo autor (2017)

Em nosso web.xml adicionamos <security-constraint> para comportar todas as configurações de segurança. Esse elemento possui dois elementos filhos importantes:

- <web-resource-collection>

Define a URL que será protegida <url-pattern>/*</url-pattern>

- <auth-constraint>

Define as roles que um usuário tem que ter para acessar a URL protegida <role-name>*</role-name>

Como queremos proteger todas as URLs da API, utilizamos o caractere curinga /*.

```
        <security-constraint>
              <web-resource-collection>
                    <web-resource-name>Controle</web-resource-name>
                    <url-pattern>/*</url-pattern>
              </web-resource-collection>
              <auth-constraint>
                    <role-name>*</role-name>
              </auth-constraint>
        </security-constraint>
```

Código-fonte 3.28 – security-constraint em web.xml
Fonte: Elaborado pelo autor (2017)

Quanto às roles, fizemos o mesmo, utilizamos o curinga * para definir que qualquer role adicionada pela tag <security-role> poderá se autenticar na URL /*.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
<security-role>
      <role-name>user</role-name>
</security-role>
<security-role>
      <role-name>guest</role-name>
</security-role>
```

Código-fonte 3.29 – security-role em web.xml
Fonte: Elaborado pelo autor (2017)

Perceba que, por meio dessa tag, acrescentamos duas roles que criamos no tomcat-user.xml. A role guest (que só acessará métodos de leitura) e a role user (que acessará os demais métodos). Essas configurações serão feitas em nosso Web Service, mais à frente.

Até esse ponto as configurações sobre as quais comentamos apenas protegeram as URLs? Precisamos de um mecanismo para exibir uma tela de login, para que o cliente possa informar o username e o password que serão confrontados pelo Tomcat, com os usuários contidos no tomcat-user.xml.

Para isso, incluímos a seguinte tag:

```
<login-config>
      <auth-method>BASIC</auth-method>
      <realm-name>ControleMemoryRealm</realm-name>
 </login-config>
```

Código-fonte 3.30 – login-config em web.xml
Fonte: Elaborado pelo autor (2017)

Com o <auth-method> configurado como BASIC, o servidor exibirá um pop-up quando o cliente tentar acessar uma URL protegida via browser. Nessa janela, ele poderá digitar o nome do usuário e uma senha. O nome de usuário e a senha serão codificados pelo algoritmo Base64 e adicionados ao cabeçalho de requisição Authorization.

Authorization: Basic YWRtaW46ZmlhcEFkbWlu

- Em que Authorization é o campo do cabeçalho HTTP.
- Basic YWRtaW46ZmlhcEFkbWlu é o valor deste campo.
- YWRtaW46ZmlhcEFkbWlu é a codificação do usuário e a senha pelo algoritmo Base64 - Base64(admin:fiapAdmin).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

O cabeçalho de requisição Authorization será utilizado pelo Tomcat, para verificar as credenciais contra a fonte de dados de usuários, configurada no nosso caso, no arquivo tomcat-users.xml. O nome do domínio de autenticação (ControleMemoryRealm) será exibido na janela de login.

Pois bem, vimos nos passos anteriores como configurar usuários, senhas e roles no arquivo tomcat-users.xml, como configurar o contexto da aplicação para o container do Tomcat utilizar o MemoryRealm em nossa API e aprendemos a configurar a aplicação, definindo as URLs protegidas, roles utilizadas para acessá-las e a tela de login para o cliente no web.xml. Tratamos autenticação, mas em momento algum definimos o que cada role pode fazer.

3.3.12.3 Autorização MemoryRealm

Com Jersey, podemos definir o acesso aos métodos da nossa API com base nas roles dos usuários, usando anotações do pacote javax.annotation.security em nossa classe UsuarioService.

Usando anotações podemos, por exemplo, definir que apenas a role "user" poderá executar um POST. Para fazermos isso, primeiro precisamos alterar a classe ServicesConfig.java para que ela registre no construtor, um filtro de requisições provido pela classe RolesAllowedDynamicFeature.

```
package br.com.fiap.controle.services;

import org.glassfish.jersey.server.ResourceConfig;
import org.glassfish.jersey.server.filter.RolesAllowedDynamicFeature;

public class ServicesConfig extends ResourceConfig  {
      public ServicesConfig() {
        super(UsuarioService.class);
        packages("br.com.fiap.controle.services");
        register(RolesAllowedDynamicFeature.class);
    }
}
```

Código-fonte 3.31 – ServicesConfig.java
Fonte: Elaborado pelo autor (2017)

Uma vez que a classe RolesAllowedDynamicFeature está registrada, vamos adicionar as anotações definidas pelo JSR-250 do javax.annotation.security em nossa API. Altere o código da classe UsuarioService.java, conforme código abaixo.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```java
package br.com.fiap.controle.services;

import java.net.URI;

import javax.annotation.security.PermitAll;
import javax.annotation.security.RolesAllowed;
import javax.persistence.PersistenceException;
import javax.ws.rs.Consumes;
import javax.ws.rs.DELETE;
import javax.ws.rs.GET;
import javax.ws.rs.POST;
import javax.ws.rs.PUT;
import javax.ws.rs.Path;
import javax.ws.rs.PathParam;
import javax.ws.rs.Produces;
import javax.ws.rs.WebApplicationException;
import javax.ws.rs.core.MediaType;
import javax.ws.rs.core.Response;
import javax.ws.rs.core.Response.Status;
import javax.ws.rs.core.UriBuilder;

import br.com.fiap.controle.dao.UsuarioDAO;
import br.com.fiap.controle.dao.impl.UsuarioDAOImpl;
import br.com.fiap.controle.entity.Usuario;
import br.com.fiap.controle.entity.wrapper.Usuarios;
import br.com.fiap.controle.exception.DataBaseException;
import br.com.fiap.controle.exception.EntityNotFoundException;

@Path("/controle")
@PermitAll
@Produces({ MediaType.APPLICATION_XML, MediaType.APPLICATION_JSON })
@Consumes({ MediaType.APPLICATION_XML, MediaType.APPLICATION_JSON })
public class UsuarioService {

	private UsuarioDAO usuarioDAO = new UsuarioDAOImpl();

	@GET
	@Path("/ola")
	@Produces("text/plain;charset=utf-8")
	public String testaServico() {
		return "Olá, o serviço funciona";
	}

	@RolesAllowed("guest")
	@GET
	@Path("/usuarios")
	public Usuarios buscarTodosUsuarios() {

		try {
			Usuarios usuarios = new
			 Usuarios(usuarioDAO.buscarTodos());
			return usuarios;
		} catch (EntityNotFoundException e) {
			throw new WebApplicationException(404);
		} catch (Exception e) {
			throw new WebApplicationException(500);
		}
	}

	@RolesAllowed("guest")
	@GET
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
	@Path("/usuarios/{id}")
	public Usuario buscarUsuarioPorId(@PathParam("id") Integer id) {
		try {
			return usuarioDAO.buscarPorId(id);
		} catch (EntityNotFoundException e) {
			throw new
			 WebApplicationException(Status.NOT_FOUND);
		} catch (Exception e) {
			throw new
			 WebApplicationException(Status.INTERNAL_SERVER_ERROR);
		}
	}

	@RolesAllowed("user")
	@DELETE
	@Path("/usuarios/{id}")
	public void removerUsuario(@PathParam("id") Integer id){
		try {
			usuarioDAO.remover(id);
		}catch (EntityNotFoundException e) {
			throw new WebApplicationException(Status.NOT_FOUND);
		}catch (DataBaseException e) {
			throw new
			 WebApplicationException(Status.INTERNAL_SERVER_ERROR);
		}
	}

	@RolesAllowed("user")
	@POST
	@Path("/usuarios")
	public Response criarUsuario(Usuario usuario){
		try {
			Usuario u = usuarioDAO.adicionar(usuario);
			URI uri =
				UriBuilder.fromPath("usuarios/{id}")
					.build(u.getIdUsuario());

			return Response.created(uri).entity(u).build();
		}catch (DataBaseException e) {
			throw new WebApplicationException(Status.CONFLICT);
		}catch (Exception e) {
			throw new
			 WebApplicationException(Status.INTERNAL_SERVER_ERROR);
		}
	}

	@RolesAllowed("user")
	@PUT
	@Path("/usuarios/{id}")
	public Usuario atualizarUsuario(@PathParam("id") Integer id,
	                                                Usuario usuario){
		try {
			buscarUsuarioPorId(id);
			usuario.setIdUsuario(id);
			return usuarioDAO.atualizar(usuario);
		}catch (WebApplicationException WebException) {
			throw WebException;
		}catch(PersistenceException PerException) {
			throw new WebApplicationException(Status.CONFLICT);
		}catch (Exception e) {
			throw new
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
                    WebApplicationException(Status.INTERNAL_SERVER_ERROR);
                }
            }
}
```

Código-fonte 3.32 – UsuarioService.java
Fonte: Elaborado pelo autor (2017)

Veja que a classe recebeu a anotação @PermitAll, para que qualquer método dela, que não possua a anotação @RolesAllowed, fique acessível para requisições de qualquer role parametrizada no web.xml da aplicação. No nosso exemplo, a anotação @PermitAll só se aplica ao método testaServico(). Todos os demais métodos da nossa API possuem a anotação @RolesAllowed, ou para a role guest ou para user.

A role guest ficou restrita aos métodos GET e a role user, restrita aos métodos POST, PUT e DELETE. Vamos testar nossa API com segurança?

3.3.12.4 Testando Autenticação e Autorização

Com o serviço executando no Tomcat, abra uma nova aba no Postman e configure a requisição GET abaixo. Na aba Authorization em Type, selecione Basic Auth e uma tela para digitação do username e password será apresentada.

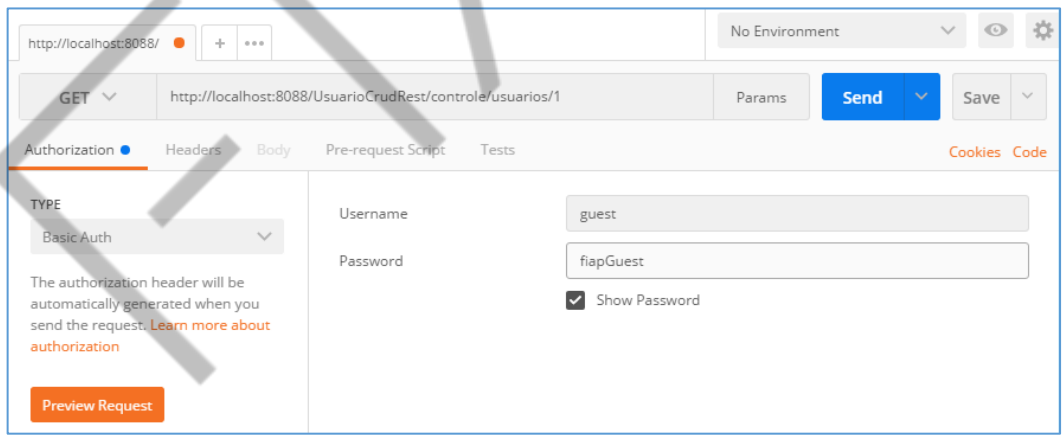

Figura 3.49 – Requisição GET – Basic Auth no Postman
Fonte: Elaborado pelo autor (2017)

Preencha os valores, clique em Preview Request e o valor do cabeçalho Authorization será preenchido antes do envio da requisição. Você poderá visualizá-lo na aba Headers. O value conterá o texto Basic mais o texto guest:fiapGuest, codificado no algoritmo Base64, resultando Z3Vlc3Q6ZmlhcEd1ZXN0.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

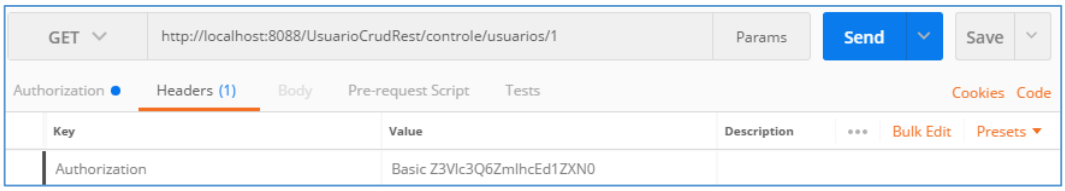

Figura 3.50 – Requisição GET – HTTP Headers com Basic Auth no Postman
Fonte: Elaborado pelo autor (2017)

Envie a requisição e o usuário de id 1 será retornado na representação XML, com o Status Code 200 OK.

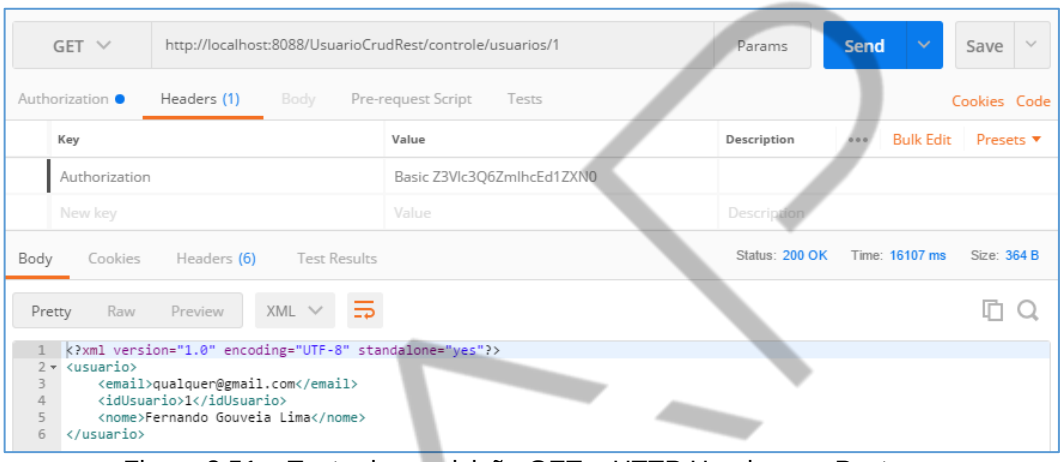

Figura 3.51 – Teste de requisição GET – HTTP Headers no Postman
Fonte: Elaborado pelo autor (2017)

Se você tentar realizar a mesma requisição GET no Mozilla Firefox, a seguinte tela de login será apresentada.

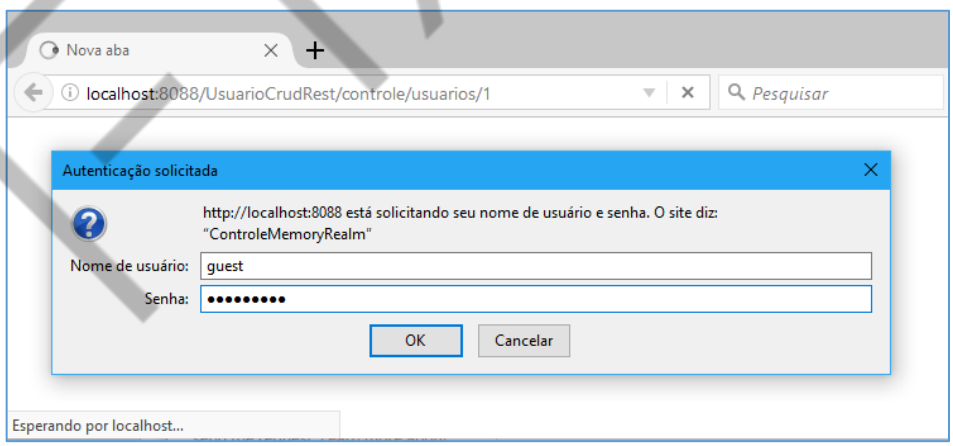

Figura 3.52 – Teste de requisição GET com MemoryRealm no Mozilla Firefox
Fonte: Elaborado pelo autor (2017)

Requisições enviadas sem o cabeçalho HTTP Authorization ou com o cabeçalho contendo usuário e senha inválidos, retornarão o Status Code 401 Unauthorized. Abaixo, tentamos o mesmo GET, sem enviar usuário e senha.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

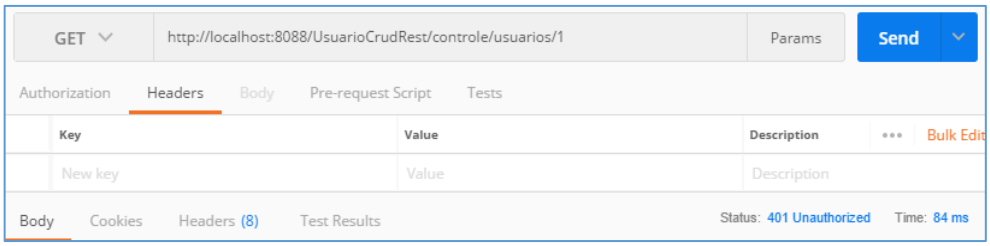

Figura 3.53 – Teste de requisição GET sem Headers com MemoryRealm no Postman
Fonte: Elaborado pelo autor (2017)

Já requisições com um usuário e senha válidos, mas sem possuir a role igual à anotação feita no método da API, retornarão o Status Code 403 (Forbidden).

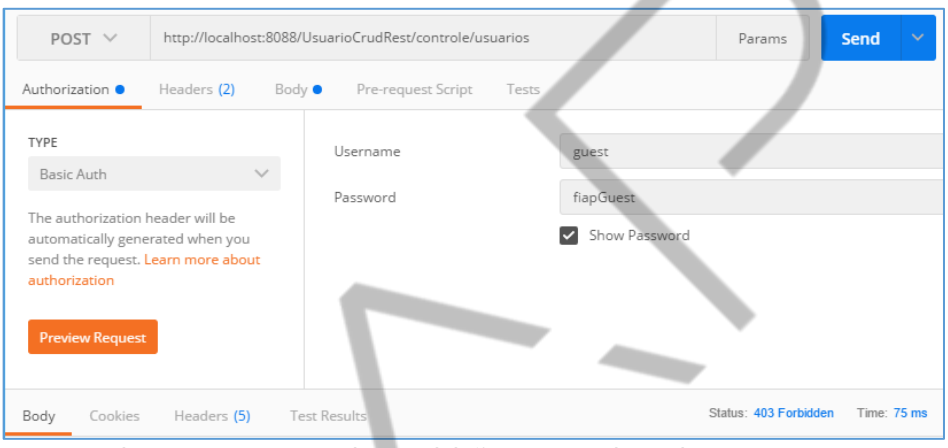

Figura 3.54 – Teste de requisição com Basic Auth no Postman
Fonte: Elaborado pelo autor (2017)

No exemplo acima, um usuário guest (de role guest) que só possui anotações @RolesAllowed em métodos GET, tentou fazer um POST e recebeu o Status 403.

Como no arquivo tomcat-users.xml definimos o usuário admin com as roles guest e user, ele tem acesso total aos métodos da nossa API.

Agora que já fizemos alguns testes e demonstramos como enviar requisições com cabeçalho Authorization, para praticar, crie uma nova Collection no Postman, faça e salve os testes da API, atendendo às configurações de security implementadas nos métodos do nosso serviço.

Bons estudos!

3.4 Considerações finais

O objetivo deste conteúdo foi abordar o básico e essencial sobre o estilo REST, aplicações RESTful, JAX-RS, Jersey e consumo do serviço com Postman. Ainda veremos Rest em um próximo capítulo e, nele, desenvolveremos clientes em Java.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Mesmo assim, existem diversos pontos a serem explorados, como o envio e o retorno de conteúdos binários, a implementação de links HATEOAS, o uso de Cache, configurar SSL no Tomcat, conhecer os demais componentes que implementam a interface Realm, entre outros.

Portanto, existem diversos conteúdos adicionais que são abordados nos livros e URLs de referência deste capítulo. Para mais informações, recomendamos a leitura.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

APACHE. Apache Tomcat 8 Configuration Reference - **The Realm Component**. [s.d.]. Disponível em: <https://tomcat.apache.org/tomcat-8.0-doc/config/realm.html>. Acesso em: 02 nov. 2017.

BOOTH David; HAAS, Hugo; MCCABE, Francis; NEWCOMER, Eric; CHAMPION Michael; FERRIS Chris; ORCHARD, David. **Web Services Architecture Note 11 February 2004.** [s.d.]. Disponível em: <https://www.w3.org/TR/ws-arch/>. Acesso em: 28 jun. 2017.

BUCEK, Pavel; PERICAS-GEERTSEN, Santiago. **JAX-RS: Java™ API for RESTful Web Services. 2017**. Version 2.1 Final Release, 13 jul. 2017.

CHAMPION, Michael. (until March 2003), Software AG, Chris Ferris (until March 2003), IBM, David Orchard (until March 2003), BEA Systems. **Web Services Architecture Note 11 February 2004.** [s.d.]. Disponível em: <https://www.w3.org/TR/ws-arch/>. Acesso em: 03 nov. 2017.

CIELO, **Tutorial Postman.** [s.d.]. Disponível em: <https://developercielo.github.io/Tutorial/Postman>. Acesso em: 02 nov. 2017.

CODE BREEZE!, **Response for GET/POST/PUT/DELETE in REST web service**. [s.d.]. Disponível em: < http://shengwangi.blogspot.com.br/2016/02/response-for-get-post-put-delete-in-rest.html>. Acesso em: 02 nov. 2017.

FIELDING, Roy T.; TAYLOR, Richard N. **Architectural styles and the design of network-based software architectures**. Doctoral dissertation: University of California, Irvine, 2000.

_____. Roy; RESCHKE, Julian. **Hypertext Transfer Protocol (HTTP/1.1)**: Authentication, 2014.

FREED, Ned; KLENSIN, J.; HANSEN, Tony. **Media type specifications and registration procedures**, 2013.

GONÇALVES, Antônio. **Introdução à Plataforma Java EE6 com GlassFish**. Rio de Janeiro : Ciência Moderna, 2011.

JCP, **JSR-000370 JavaTM API for RESTful Web Services (JAX-RS).** [s.d.]. Disponível em: <https://jcp.org/aboutJava/communityprocess/final/jsr370/index.html>. Acesso em: 02 nov. 2017.

_____. **JSR 250: Common Annotations for the JavaTM Platform**. [s.d]. Disponível em: <https://jcp.org/en/jsr/detail?id=250>.Acesso em: 02 nov. 2017.

JERSEY, **Jersey 2.26 - User Guide.** [s.d.]. Disponível em: <https://jersey.github.io/documentation/latest/index.html>. Acesso em: 02 nov. 2017.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

_____. **Jersey 2.26 - Chapter 16. Security**. [s.d.]. Disponível em: <https://jersey.github.io/documentation/latest/security.html>. Acesso em: 02/11/2017.

LECHETA, Ricardo R. **Web Services RESTful:** Aprenda a criar web services RESTful em Java na nuvem do Google. Novatec Editora, 2015.

MOZILLA, MDN web docs, **HTTP authentication.** [s.d.]. Disponível em: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Authentication>. Acesso em: 02 nov. 2017.

_____. MDN web docs, **Hypertext Transfer Protocol (HTTP).** [s.d.]. Disponível em: <https://developer.mozilla.org/en-US/docs/Web/HTTP>. Acesso em: 02 nov. 2017.

ORACLE, 32 Building RESTful Web Services with JAX-RS. Disponível em: < https://javaee.github.io/tutorial/jaxrs.html>. Acesso em: 02 nov. 2017.

RESCHKE, J.; FIELDING, Roy. RFC 7231-Hypertext Transfer Protocol (HTTP/1.1): **Semantics and Content**. Jun 2014. Disponível em: <http://tools.ietf. org/html/rfc7231>. Acesso em: 19 fev. 2015.

REST API TUTORIAL, **Learn REST: A RESTful Tutorial.** [s.d.]. Disponível em: <http://www.restapitutorial.com/>. Acesso em: 02 nov. 2017.

RESTCOOKBOOK.**What is HATEOAS and why is it important for my REST API?**. [s.d.]. Disponível em: <http://restcookbook.com/Basics/hateoas/>. Acesso em: 02 nov. 2017.

RICHARDSON, Leonard; RUBY, Sam. **RESTful web services**. " O'Reilly Media, Inc.", 2008.

RODRIGUEZ, Alex. Restful web services: **The basics.** IBM developerWorks, 2008.

SAMPAIO, Cleuton. Java Enterprise Edition 6- **Desenvolvendo Aplicações Corporativas**. Brasport, 2011.

SANDBOX, **Sandbox Definition API.** [s.d.]. Disponível em: <https://getsandbox.com/docs/definition-api>. Acesso em: 02 nov. 2017.

SAUDATE, Alexandre. REST: **Construa API's inteligentes de maneira simples**. Editora Casa do Código, 2014.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

GLOSSÁRIO

REST	Representational State Transfer (REST) é um estilo arquitetônico que especifica restrições de design, para que aplicações que implementem este estilo aproveitem a arquitetura Web em seu benefício. Traduzindo livremente “Representational State Transfer” ou REST, temos Transferência de Estado Representativo. O termo REST foi introduzido e definido em 2000, na tese de doutorado de Roy Fielding para a University of California, IRVINE.
HTTP	O Hypertext Transfer Protocol, sigla HTTP (em português Protocolo de Transferência de Hipertexto) é um protocolo de comunicação (na camada de aplicação segundo o Modelo OSI), utilizado para sistemas de informação de hipermídia, distribuídos e colaborativos. Ele é a base para a comunicação de dados da World Wide Web.
URI	Uniform Resource Identifier (URI) é uma sequência de caracteres usados para identificar um recurso. Essa identificação permite a interação com representações do recurso através de uma rede, tipicamente na World Wide Web, usando protocolos específicos.
RESTful	Ao se enquadrar nas restrições REST, uma aplicação se torna RESTful e obterá propriedades desejáveis no ambiente Web, tais como, comunicação heterogênea entre soluções (com baixa complexidade semântica), maior performance, escalabilidade e facilidade de desenvolvimento e manutenção.
JSON	JSON (JavaScript Object Notation - Notação de Objetos JavaScript) é uma formatação leve de troca de dados. Para seres humanos, é fácil de ler e escrever. Para máquinas, é fácil de interpretar e gerar. Está baseado em um subconjunto da linguagem de programação JavaScript, Standard ECMA-262 3a Edição - Dezembro – 1999
Swagger	A Swagger é um framework para desenvolvimento de APIs para a especificação OpenAPI (OAS), permitindo o desenvolvimento em todo o ciclo de vida da API, desde o design, documentação, testes e a implantação. https://swagger.io/

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

API Blueprint	API Blueprint é uma poderosa linguagem de descrição para APIs da web. API Blueprint é simples e acessível a todos os envolvidos no ciclo de vida da API. https://apiblueprint.org/
WSDL	A Web Services Description Language (WSDL) é uma linguagem baseada em XML utilizada para descrever Web Services funcionando como um contrato do serviço. Trata-se de um documento escrito em XML que além de descrever o serviço, especifica como acessá-lo e quais as operações ou métodos disponíveis.
RAML	RESTful API Modeling Language (RAML) é um idioma baseado em YAML para descrever as APIs RESTful . O RAML fornece todas as informações necessárias para descrever APIs RESTful. https://raml.org/
JSONView	Com o complemento JSONView, documentos JSON são mostrados nos navegadores do mesmo modo como documentos XML. O documento é formatado, realçado e objetos podem ser recolhidos. Mesmo que o documento JSON contenha erros, o JSONView mostrará ainda o texto puro.
SOAP	SOAP (Simple Object Access Protocol, em português Protocolo Simples de Acesso a Objetos) é um protocolo para troca de informações estruturadas em uma plataforma descentralizada e distribuída. Ele se baseia na Linguagem de Marcação Extensível (XML) para construir seu formato de mensagem, e normalmente baseia-se em outros protocolos da camada de aplicação, mais notavelmente em chamada de procedimento remoto (RPC) e Protocolo de transferência de hipertexto (HTTP), para negociação e transmissão de mensagens.
XML	XML é uma recomendação da W3C para gerar linguagens de notação para necessidades especiais. É um dos subtipos da SGML capaz de descrever diversos tipos de dados
Firewall	Um firewall é um dispositivo de uma rede de computadores que tem por objetivo aplicar uma política de segurança a um determinado ponto da rede. O firewall pode ser do tipo filtros de pacotes, proxy de aplicações, etc.
Marshalling	Marshalling é o processo de transformação da representação de memória de um objeto em um formato de dados compatível para armazenamento ou transmissão (os formatos mais comuns são XML e JSON). É usado

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

	tipicamente quando dados precisam ser transferidos entre diferentes partes de um aplicativo de computador ou de um aplicativo para outro.
Unmarshalling	Unmarshalling é o processo reverso do Marshalling, ou seja, de transformar uma representação de um objeto que foi usado para armazenamento ou transmissão (em XML ou JSON) para uma representação do objeto executável em memória.
CSV	Os arquivos Comma-separated values, também conhecido como CSV, são arquivos de texto de formato regulamentado pelo RFC 4180, que fazem uma ordenação de bytes ou um formato de terminador de linha.
Deflate	O algoritmo DEFLATE é usado nos utilitários que comprimem ou descomprimem arquivos no padrão ZIP ou no padrão gzip. O formato ZIP se tornou popular por meio do programa PKZIP, e é hoje usado na maioria dos programas de compressão de dados.
Postman	O Postman é um Client de APIs que facilita aos desenvolvedores criar, compartilhar, testar e documentar APIs. Isso é feito, permitindo aos usuários criar e salvar solicitações HTTP / s simples e complexas, bem como ler suas respostas.
Mock	Objetos Mock, objetos simulados ou simplesmente Mock (do inglês Mock object) em desenvolvimento de software são objetos que simulam o comportamento de objetos reais de forma controlada. São normalmente criados para testar o comportamento de outros objetos. Em outras palavras, os objetos mock são objetos “falsos” que simulam o comportamento de uma classe ou objeto “real” para que possamos focar o teste na unidade a ser testada.
Maven	Apache Maven, ou Maven, é uma ferramenta de automação de compilação utilizada primariamente em projetos Java.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com